

The Compositional Essence of Effectful Cost Analyses*

Categorical Foundations and Fibered Logical Relations

PEDRO H. AZEVEDO DE AMORIM, University of Bath, UK

This paper introduces a framework based on category theory and denotational semantics for the compositional analysis of effectful notions of cost. Over the past few years there has been a proliferation of language-based techniques for reasoning about cost in the presence of effects, such as expected cost analysis for probabilistic effects, and best/worst-case cost for non-deterministic effects.

A significant limitation of existing approaches is that each cost analysis must be done from scratch and their soundness must be established for each individual case. This specialized approach makes it harder to exploit the high-level structures behind such analyses.

In this work we mitigate these issues by proposing **effcost**, a call-by-push-value (CBPV) metalanguage for effectful cost analyses, and showing that it has a simple and general categorical semantics. We justify its soundness by comparing it to two known approaches from the literature: using continuation-passing-style (CPS) and the writer monad transformer. Both soundness proofs follow from a categorical logical relations argument. In order to prove one of these soundness properties we further develop the theory of categorical logical relations so that they can handle approximate notions of logical relations.

1 INTRODUCTION

This paper is about the foundations of effectful cost analysis. By effectful cost analysis, we mean the emergent notions of cost that appear when one considers the interaction between cost and computational effects. An illustrative example is non-determinism, where there are various natural cost metrics of interest, such as best cost, worst cost, every set of cost, etc.

While much work has been done in giving different compositional accounts to different kinds of effectful cost analysis [18, 32, 45, 61, 65, 75], there is no general theory of when such analyses are valid. This hinders the adoption of such ideas, as whenever one wants to consider a new kind of effectful cost analysis, they must first establish soundness and compositional properties of their novel analysis from scratch.

The interest in this goes beyond purely theoretical. As new programming abstractions and models become mainstream, or existing applications are further specialized, new notions of cost become relevant. For instance, with the advances of probabilistic programming over the last decade [10] there has been considerable effort in developing techniques for reasoning about the expected cost of probabilistic programs [7, 38, 59, 72, 75].

In terms of relevant previous work, there are two kinds of general approaches that are most relevant to this paper. The first one is the **decalf** type theory [26], which uses the writer monad transformer $T(\mathbb{C} \times -)$ to reason about costful and effectful programs, where $\mathbb{C}$ is a cost monoid and T is your base monad. The other approach is to repurpose continuation-passing-style (CPS) transformations so that they can incorporate cost information as well [2, 4, 37, 60].

While both approaches have expanded our understanding of effectful cost analyses, they still do not get to the core of what makes an effectful cost analysis sound and compositional. Going back to the non-determinism case, suppose that instead of more traditional cost analysis, we have a new metric for cost. What are the properties that guarantee that such a metric is compatible is compositional reasoning? Existing approaches do not have a way of answering this question.

*For the purpose of Open Access the author has applied a CC BY public copyright licence to any Author Accepted Manuscript version arising from this submission.

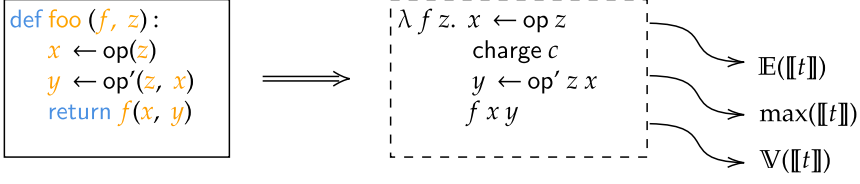


Fig. 1. Source-to-semantics-to-cost schematic

Ideally, we want our formalism to be parametric in the base effect, effectful notion of cost analysis, cost models and evaluation strategies. In Figure 1, we see a schematic of what we want: potentially different source languages being elaborated into a core metalanguage, with cost annotations added — the operation charge c in the figure. Then, depending on the semantic domain and on the interpretation of the effectful operations op , the semantics would target different kinds of cost analysis. While the task of developing such core metalanguages is relatively well-understood, much work remains to be done on the second component.

1.1 Desiderata: Compositional Methods in Effectful Cost Analysis

Like many problems in PL theory, there may be multiple approaches to effectful cost analyses. In order to circumscribe the state space of possible solutions, we propose two reasonable desiderata on our sought-after theory. The guiding design principles behind it are generality and simplicity.

This improves on the status quo, which mostly focuses on studying specific models in isolation, with a couple of exceptions [26, 48]. As a consequence, while we do have a robust set of examples of compositional and sound cost analyses, we do not know why they work at a more fundamental level. Given this well-established but still incomplete body of work, we have the following desideratum.

Desideratum 1. Our theory should encompass concrete existing examples and propose sufficient conditions for a new cost analysis to be sound and compositional.

In the writer monad transformer approach, while very general, it cannot compositionally reason about effectful notions of cost. Consider once again worst cost analysis for non-deterministic computations. By interpreting programs as $\mathcal{P}(\mathbb{N} \times A)$, where $\mathcal{P}$ is the powerset monad, the worst cost must be computed from a powerset whose number of elements grows quadratically.

The CPS approach also violates this desideratum. Indeed, as of now, we do not have sufficient conditions on what makes a particular cost metric compatible with CPS reasoning. The difficulty in coming up with new CPS translations is high enough that they often are a significant contribution in and of itself [4, 12, 37, 38].

Another issue with CPS semantics is that they agglutinate control flow, costful and effectful aspects of program execution into the same program transformation [48]. This leads us to the next desideratum.

Desideratum 2. There should be a *semantic* separation between the cost semantics and the effectful semantics.

Modularity is a cornerstone methodology in the formal verification and PL communities that enables the scalability of program analyses. Therefore, we want our theory to satisfy modular reasoning.

In contrast to existing work where modularity may be established a posteriori [5], we want our theory to validate them by construction. It should be mathematically impossible for our theory to not be modular. Concretely, we want the functional aspects of programs to be completely orthogonal to their costful properties. Effectively, *the cost semantics should be a refinement of the base semantics*.

1.2 Our Solution: Categorical Semantics of Effectful Cost Analysis

We present **effcost**: a call-by-push-value [52] (CBPV) metalanguage with a compositional effectful cost semantics. By having the full expressive power of CBPV, **effcost** can handle different evaluation strategies and higher-order functions while being parametric on base effects and notions of effectful cost. Our metalanguage and its categorical semantics is the first such language to validate all of the desiderata above:

Desideratum 1: In Section 2 we introduce *effectful monoids*. From them we can extract sufficient conditions on when effectful cost analyses can be made compositional. In Section 3 we show that this abstract semantics can very naturally capture useful examples such as expected cost, higher-moments, worst/best-case cost and weighted cost. Furthermore, by being based on CBPV, both imperative and functional languages with different evaluation strategies can be embedded into **effcost**.

Desideratum 2: In Section 2.1, we show that **effcost**'s semantics has a very clear separation between its costful and non-costful components, allowing for a modular approach to cost analysis.

It is important to highlight the central role that category theory plays in this paper. By formulating our problem categorically and using established tools from the categorical study of programming languages, we can follow the “path of least resistance” and obtain a general and simple semantics with an intuitive interpretation.

Categorical reasoning also provides powerful tools for formally relating our semantics with the CPS and monad transformers ones. For the sake of understanding the complex tapestry of different approaches to compositional cost analyses, it is useful to show that different cost semantics are essentially reasoning about the same property.

Finally, we connect our work to recent advances on categorical semantics. For example, in Section 6, we exploit the cost/effect separation to soundly extend **effcost** with parallel execution. This was one of the open problems left by Grodin et al. [26].

1.3 Results and Organization

effcost is a simple extension of CBPV with cost operations. Its main advantages over similar calculi are its generality in terms of which analyses and programming paradigms it supports, and the simplicity of its semantics. We also show that **effcost** can be naturally interpreted with a CPS semantics and a writer monad transformer semantics.

Somewhat surprisingly, even though our semantics is quite simple, we show that it captures the essence of CPS-based approaches to cost analysis in a sense we make precise in Theorem 5.1. Therefore, even though someone might care about CPS due to, for instance, implementation reasons, our semantics can also be used to guide the design of new CPS-based program analyses.

While the proof of Theorem 5.1 follows from somewhat standard tools from fibered logical relations, the soundness of the monad transformer requires significant extensions to the categorical study of logical relations. We achieve this by introducing and proving structural theorems about lax fibrations, a generalization of a concept widely used programming language theory [42, 44]. We believe that these results are of broader interest because such fibrations allow for the abstract construction of new classes of logical relations for establishing approximation relations across different semantics.

In summary, the main contributions of this paper are the following.

- We define effectful cost monoids. The semantics of **effcost** will be defined using lifts of the writer monad of effectful monoids to categories of algebras of monads (§2.1).

- We develop a new fibred account to approximate logical relations through the development of the theory of lax fibrations (§4.2).
- We prove the soundness of the cost semantics of **effcost** with respect to a CPS semantics and use our new notion of approximate logical relation to establish the soundness of our semantics with respect to the monad transformer semantics (§5).
- We give sufficient conditions on our base effect and cost analysis for it to be compatible with parallel execution through the use of concurrent (pseudo)monads (§6).

Organization of the paper. In Section 2, we present the syntax and semantics of **effcost**. It is a CBPV calculus extended with inductive types and parametric in a set of effectful operations. Its semantics is based on the algebraic semantics of CBPV first introduced by Levy [52, §12]. In Section 3, we show how to instantiate our categorical semantics to concrete instances of effectful notions of cost, illustrating the versatility of **effcost**. In Section 4, we set up the necessary category theoretic machinery powering our soundness proofs and in Section 5 we deploy it to prove our soundness theorems. We conclude the technical development of our work in Section 6, where we propose a refinement of our base semantics and sufficient conditions on our effect and notion of cost so that it can also accommodate parallel notions of effectful cost.

The category theory used varies throughout the paper. For the definition of **effcost** and its semantics we do not go beyond standard notions of category theory used in programming language research, such as Cartesian closed categories, monads and their algebras. For the construction of our logical relations we make use of notions coming from fibred category theory that are standard in categorical formulations of logical relations [30, 39, 40, 42, 44].

2 **effcost**: MODULAR COST AND EFFECTS SEMANTICS

In this section we introduce the syntax of **effcost**, an extension of CBPV for effectful cost analysis. Then, we define its denotational semantics based on *effectful monoids*, an abstraction for monoids that interact well with the underlying effect. We conclude this section by proposing other cost semantics of **effcost** that are based on monad transformers and CPS semantics.

The syntax of **effcost** is depicted in Figure 2. As usual for CBPV, types are separated into value types and computation types with modalities making both grammars interdependent. The modality U and the operations **thunk** and **force** provide fine-grained control over when to stop or resume the execution of a computation. The F modality is for computations that eventually return a value. The program **produce** V represents the trivial computation that just outputs V . Computations can be chained together using $x \leftarrow t; u$. The other connectives and their corresponding syntax are common in non-effectful programming languages and can be found in standard textbooks [28]. It's worth noting that the arrow type has a value type as its domain and a computation type as its codomain, which is why applications are between a computation and a value. Lastly, note how there are two different kinds of product types. The computation product types are *lazy* products with projections as eliminators while value product types are *eager*, with sequencing as eliminators.

We assume a set of basic value types $\mathcal{B}$ and effectful operations $\mathcal{O}$. In order to simplify our technical development, we will assume that our language does not have recursion primitives. That being said, they are easy to incorporate within CBPV and its semantics [45, 52]. The full typing rules can be found in the appendix.

Cost as an effect. In cost-aware languages, it is common to model cost as an effectful operation that allows you to instrument your program so that it is independent of a fixed cost model [2, 19, 32, 61]. Syntactically, we assume the existence of a value type $\mathbb{A}$ and an operation **charge** $: \mathbb{A} \rightarrow F1$ such that **charge** V increases the cost of the computation in V units. Semantically, such operations are modeled using monoids and monoid actions, which are presented by the writer monad $\mathbb{A} \times -$.

Value Types	τ	$\equiv$	$U\bar{\tau} \mid \alpha \mid 1 \mid b \in B \mid \mathbb{A} \mid \tau + \tau \mid \tau \times \tau \mid \mu\alpha.\tau$
Computation Types	$\bar{\tau}$	$\equiv$	$F\tau \mid \tau \rightarrow \bar{\tau} \mid \bar{\tau} \times \bar{\tau}$
Computation Terms	t, u	$\equiv$	$\lambda x. t \mid t V \mid \text{force } V$ $\mid (x \leftarrow t); u \mid \text{produce } V \mid \text{let } x \text{ be } V \text{ in } t$ $\mid \text{op} \in \mathcal{O} \mid \text{let } (x, y) = V \text{ in } t \mid \text{charge } V$ $\mid t_1 \& t_2 \mid \pi_i t \mid \tau.\text{fold } t \mid \text{match } V \text{ with roll } x \rightarrow t$ $\mid \text{case } V \text{ of } (\text{in}_1 x \Rightarrow t \mid \text{in}_2 x \Rightarrow u)$
Value Terms	V	$\equiv$	$x \mid () \mid b \in B \mid a \in \mathbb{A} \mid \text{thunk } t \mid (V_1, V_2)$ $\mid \tau.\text{roll } V \mid \text{in}_1 V \mid \text{in}_2 V$

Fig. 2. Types and Terms of CBPV

Inductive types. We also assume the presence of (positive) inductive types. There are type variables α and fixed points $\mu\alpha.\tau$, where α only appears in positive position. Once again, we have decided against including full recursive types, as they provide unnecessary complications to the semantics. In the following example we show how, even without arbitrary recursion, we can still exploit structural recursion to write nontrivial programs. Throughout this paper we do not use the primitives `fold` and `match` and instead we will use syntactic sugar to programming with inductive types similar to how Haskell does it.

Example 2.1. Suppose that we have the list and binary tree datatypes, defined as the value types $\mu\alpha. 1 + \mathbb{N} \times \mathbb{N}$ and $\mu\alpha. 1 + \alpha \times \mathbb{N} \times \mathbb{N}$, respectively. Below we present a higher-order version of a function that maps a list to its binary search tree. We assume its functional argument chooses what element to insert next. Because **effcost** does not have unbounded recursion, we write a structurally recursive program by tracking a natural number n corresponding to the size of the input list.

The functional argument of the program `searchTree` is a function that chooses the next element from a list and the rest of the list. In the basic case, it simply outputs the head and tail of the list.

This strategy is susceptible to problematic inputs. That is, if the input list is ordered, the output search tree will be a straight line. In Section 3 we show how for probabilistic effects there is a better choice for the selection function. The function `insert` inserts a new element into its correct position in a binary search tree.

```

searchTree :  $\mathbb{N} \rightarrow [\mathbb{N}] \rightarrow U([\mathbb{N}] \rightarrow F(\mathbb{N} \times [\mathbb{N}])) \rightarrow F(\text{Tree}_{\mathbb{N}})$ 
searchTree 0 l f = Empty
searchTree (n + 1) Nil f = Empty
searchTree (n + 1) l f =
  (x, xs)  $\leftarrow$  (force f) l
  b  $\leftarrow$  searchTree n xs f
  insert x b

```

2.1 Denotational Semantics

We now give a denotational semantics to **effcost**. We begin by recalling the definition of strong monads.

Definition 2.2 ([58, Definition 1.2, Definition 2.2]). A strong monad on a bicartesian closed category $\mathcal{C}$ is a 4-tuple $(T, \eta, (-)^\#, \sigma)$ where $T : \mathcal{C} \rightarrow \mathcal{C}$ is an endofunctor, $\eta : 1 \rightarrow T$ is the unit natural

transformation, $(-)^{\#} : (- \Rightarrow T =) \rightarrow (T- \Rightarrow T =)$ is the Kleisli extension natural transformation and $\sigma : - \times T \Rightarrow T(- \times =)$ is the strength. They satisfy well-known equations.

The structure above is enough to give semantics to Moggi's metalanguage. However, the categorical semantics of CBPV is more involved [52, Part III]. Luckily, every model of Moggi's metalanguage gives rise to a canonical CBPV model by using the category of algebras of the monad. This semantics was first defined by Levy in his dissertation [52, §12] and we now present its basic definitions.

Definition 2.3. Given a monad $T : C \rightarrow C$, a T -algebra is a pair $(A : C, \alpha : TA \rightarrow A)$ such that the following diagrams commute

$$\begin{array}{ccc} A & \xrightarrow{\eta_A} & TA \\ & \searrow & \downarrow \alpha \\ & & A \end{array} \quad \begin{array}{ccc} T^2 A & \xrightarrow{\mu_A} & TA \\ T\alpha \downarrow & & \downarrow \alpha \\ TA & \xrightarrow{\alpha} & A \end{array}$$

An algebra map $(A, \alpha) \rightarrow (B, \beta)$ is a C morphism $f : A \rightarrow B$ such that $\beta \circ Tf = f \circ \alpha$. The Eilenberg-Moore category C^T has T -algebras as objects and algebra maps as morphisms.

There is a canonical *forgetful* functor $U : C^T \rightarrow C$ that maps (A, α) to A . Given a strong monad $T : C \rightarrow C$, the *algebraic* semantics of CBPV follows the following pattern:

- Value types τ denote C objects $\llbracket \tau \rrbracket_v$.
- Value terms $\Gamma \vdash_v t : \tau$ denote C morphisms $\llbracket \Gamma \rrbracket_v \rightarrow \llbracket \tau \rrbracket_v$, where the interpretation of contexts is given by the elementwise product.
- Computation types $\bar{\tau}$ denote T -algebras $\llbracket \bar{\tau} \rrbracket_c$.
- Computation terms $\Gamma \vdash_c t : \bar{\tau}$ denote C morphisms $\llbracket \Gamma \rrbracket_v \rightarrow U \llbracket \bar{\tau} \rrbracket_c$.

When it is clear from the context, we will drop the c/v indices from the semantic brackets. At first, it might seem a bit unintuitive that computations do not denote algebra maps. The reason for that is that the free variables in computation terms are of value type, not computation type.

The interpretation of the operation-free fragment of **effcost** is standard [52, §12.5] and can be found in the appendix. We illustrate how the computation types are interpreted. The algebra structure associated to the type constructor F is the monad multiplication μ , the product structure is given component wise and the arrow type makes use of the following algebra structure.

Definition 2.4. Assuming $A : C$ and (B, β) is a T -algebra, we can equip $A \rightarrow B$ with the pointwise algebra structure $\alpha_{A \rightarrow B} = A \times T(A \rightarrow B) \xrightarrow{\sigma} T(A \times (A \rightarrow B)) \xrightarrow{T(ev)} TB \xrightarrow{\beta} B$, where we implicitly use the Cartesian closed isomorphism $A \times B \rightarrow C \cong A \rightarrow (B \rightarrow C)$. The morphism ev is the application operation given by the Cartesian closed structure of C .

The cost-free, operations op are assumed to have an interpretation $\llbracket \text{op} \rrbracket^{cf} : \llbracket \Gamma \rrbracket \rightarrow T \llbracket \tau \rrbracket$. In order to interpret the costful aspects of **effcost**, we will use a generalization of the writer monad, which we now construct. We begin by defining monoid objects in Cartesian categories.

Definition 2.5. A monoid object in a Cartesian category C is an object A equipped with arrows $e : 1 \rightarrow A$ and $m : A \times A \rightarrow A$, for the unit and multiplication, respectively, such that it satisfies the monoid laws written as morphism equations.

When our underlying category is **Set**, this recovers the original definition of monoid. It is well-known that monoid objects can be used to keep track of resource usage in programs by using the writer monad $A \times -$ [45, 61]. The monoid unit and multiplication model, respectively, no resource usage and adding resource usage.

$$\begin{array}{ccc}
C & \xleftarrow{\quad} & C^T \\
& \perp & \\
& \xrightarrow{\quad} & \\
& & (C^T)_{M \times -}
\end{array}$$

Fig. 3. Stacked adjunctions

The presence of other effects complicates matters, as resource consumption may now depend on non-trivial interactions between side-effects and control-flow primitives. With this in mind, we now provide a sound approach for compositionally reasoning about such interactions. An important insight from this work is that, assuming that your base effect can be interpreted by a monad T , in order to soundly incorporate the cost information given by a monoid M with the effectful execution of your program, there should be a map $TM \rightarrow M$ that interacts well with the monad structure on T . This motivates the following definition.

Definition 2.6. A T -effectful monoid is a monoid object in C^T .

Similar definitions have been considered by Seal [69] in the context of monoidal structures on categories of algebras for commutative monads. We now can define our effectful cost monad.

Theorem 2.7. For every T -effectful monoid object M , $M \times T(-)$ can be equipped with a canonical monad structure.

PROOF. The unit is $A \xrightarrow{\langle 0, \eta \rangle} M \times TA$ and the Kleisli lifting of $f : A \rightarrow M \times TB$ is given by

$$M \times TA \xrightarrow{id \times \langle \alpha \circ T(\pi_1 \circ f), id \rangle} M \times M \times TA \xrightarrow{add \times (\pi_2 \circ f)^\#} M \times TB$$

The strength is defined as $M \times A \times TB \xrightarrow{M \times \sigma} M \times T(A \times B)$. While the arrow notation might obfuscate matters, using the internal language of CBPV the lifting looks quite intuitive: assuming that $\vdash_c t : A \rightarrow M \times FB$ and M is a computation type we define the lifting as:

$$\begin{aligned}
& \lambda x : U(M \times FA). \\
& \text{let } (m_1, y) = \text{force } x \text{ in} \\
& a \leftarrow y \\
& \text{let } (m_2, z) = t \ a \text{ in} \\
& (m_1 + m_2, z)
\end{aligned}$$

One must be careful when interpreting the term above, as it might lead the reader to mistakenly think that we are simply adding the costs m_1 and m_2 . Semantically, in the line $a \leftarrow y$ the algebra structure $TM \rightarrow M$ is applied (cf. the appendix).

The proof that this construction does indeed satisfy the monad laws follows by direct algebraic manipulations. Note that in the definitions above, **the base monad component remains unchanged** while the cost component only interacts with the cost component of f/t . $\square$

A more conceptual proof of the theorem above follows from the observation that we can equivalently describe our cost monad as the resulting monad one gets from resolving the stack of adjunctions depicted in Figure 3 — this can be equivalently described through the language of *distributive laws* of monads [13]. With this monad, we can now interpret the effectful fragment of **effcost** as follows:

$$\begin{aligned}
\llbracket \text{charge } V \rrbracket_c &= (\llbracket V \rrbracket_v, \eta(*)) \\
\llbracket \text{op} \rrbracket_c &= (0, \llbracket \text{op} \rrbracket_c^{cf}),
\end{aligned}$$

where $\llbracket - \rrbracket_c^{cf}$ is the cost free semantics. Note the clear separation between the base effect and the cost effect. This is a useful artifact when it comes to reasoning about programs, as the effectful component does not interact at all with the cost semantics.

2.2 Revisiting Two Known Models

An important insight from denotational semantics is that different models may elucidate different aspects of program execution. In the context of cost-aware models, as mentioned in the introduction, there are alternatives that have been considered before in the literature [2, 26]. While they are valuable contributions to the field, we believe that the semantics defined above provides better abstractions for reasoning about programs. This is corroborated by recent work [7, 65], where an instantiation of **effcost**'s semantics has been used to reason about programs that were outside of scope of existing techniques. We now formally present two alternative models and in Section 5 we establish formal connections between all of these semantics.

Monad transformer semantics. One notable property of the writer monad is that for every strong monad T , and every monoid object M , $T(M \times -)$ is a monad. Therefore, by choosing an appropriate cost monoid and an arbitrary monad, we can define a cost monad and use it to interpret **effcost**.

The monad transformer semantics is, in a certain sense, the “true” cost semantics. By having complete information of how the cost interacts with the underlying computation, one can use this information to analyze different aspects of the cost semantics. For instance, in the probabilistic case, once you have access to a distribution $\mathcal{D}(\mathbb{N} \times A)$, it is possible to calculate higher-moments of the cost marginal and also reason about possible correlations between costly computations and their outputs.

However, adopting such a general semantics destroys nice compositional aspects of the cost analysis present in the semantics of **effcost**. Once again, let us consider the probability example. The monad transformer cost semantics allows us to compositionally compute the cost distribution $\mathcal{D}(\mathbb{N} \times A)$, but in order to compute its expected value, we have to apply the expected cost formula to a potentially quite involved distribution. This is in stark contrast to the **effcost** semantics, where the expected value would be incrementally and compositionally computed [7, 65]. Furthermore, for certain effects this semantics validates undesirable equations, as we now illustrate.

Example 2.8. Consider the maybe/option monad $1 + -$. Applying the cost monad transformer with the $\mathbb{N}$ additive monoid gives us $1 + (\mathbb{N} \times -)$. Under this semantics, the following program equation is valid $\text{charge } 1; \text{error} = \text{error}$. Such an equation is not realistic, as it says that as long as the program outputs an error, all of the cost information is discarded. In contrast, our semantics distinguishes the programs above when we use the algebra structure $[0, \text{id}] : 1 + \mathbb{N} \rightarrow \mathbb{N}$ that associates no cost to errors, the notation $[f, g] : A + B \rightarrow C$ denotes the universal arrow given by the universal property of sum types.

Continuation passing style semantics. Many cost-aware denotational semantics are based on continuation monads $(- \rightarrow A) \rightarrow A$ with appropriate answer types A . For instance, there is a seminal line of work on using the answer type $[0, \infty]$, the so-called pre-expectation monad, to reason about expected cost [38].

There are a couple of important limitations to this approach. The first one is that, as we currently understand it, the task of finding appropriate response types for particular effects is still largely by trial-and-error. In contrast, by tightly-coupling our effectful cost monoid to the base monad, we significantly reduce the search space and provide a principled way of proposing new cost analyses.

Another limitation of using the continuation monad is that since it is such a general monad that can implement every monadic effect, it obfuscates useful invariants existing in bespoke semantics.

Effectful analysis	Computational effect	Category	Monad	Algebra structure
Expected cost[7] (cf. Section 3.1)	Probability	ωQBS	P	$\mu \mapsto \mathbb{E}(\mu)$
Higher moments (cf. Section 3.2)	Probability	Set	$\mathcal{D}$	$\mu \mapsto (\mathbb{E}(\mu_1), \dots, \mathbb{E}(\mu_n))$
Failure-insensitive cost (cf. Example 2.8)	Failure	C	$1 + -$	$\text{error} \mapsto 0$ $\text{Just } n \mapsto n$
Weighted cost (cf. Section 3.3)	Weigthed execution	Set	$- \rightarrow_{fs} \mathcal{R}$	$\varphi \mapsto \sum_r r\varphi(r)$
Worst-case expected cost (cf. Section 3.4)	Imprecise probability	Set	$\mathcal{P}_c$	$C \mapsto \max_{\mu \in C} \mathbb{E}(\mu)$

Fig. 4. Summary of different effectful cost analysis considered

Indeed, many desirable properties of effects are lost when interpreting them in a continuation-passing style fashion, commutativity being a notable example.

3 A CATALOGUE OF EFFECTFUL COST ANALYSES

In this section we go over some examples that illustrate how ubiquitous our abstract denotational semantics is. While the first model is known in the literature [7], we provide the first cost semantics to a functional programming language for higher-moments and weighted programming.

3.1 Warm-up: Expected Cost Analysis

In this example we revisit a simplified version of the expected cost model of Azevedo de Amorim [7]. Let $\mathcal{D}$ be the probability distribution monad on **Set** and assume that the language comes equipped with basic probabilistic primitives, such as coins and, for instance, an operation `unif` that uniformly samples an element from a list. We start by defining the appropriate algebra structure over a monoid object.

Definition 3.1. The positive real line $\mathbb{R}^+$ is a $\mathcal{D}$ -algebra with the algebra structure given by the expected value. Furthermore, it can be equipped with the canonical additive monoid structure.

In this case the expected cost monad interprets programs as pairs $\mathbb{R}^+ \times \mathcal{D}(\tau)$, where the $\mathbb{R}^+$ component corresponds to the expected cost of the program. Now we consider a probabilistic variant of Example 2.1. In this case, we uniformly sample elements from the list and insert them into the search tree. It is known that the expected cost of the height of randomized search trees, and therefore expected cost of insertion, is $O(n \log n)$ [21]. However, a careful semantic analysis of this cost bound is outside of scope.

3.2 Higher-Moments

While reasoning about expected values allows you to impose certain bounds on the cost distribution, it is still too coarse an approximation for certain applications [49]. In order to circumvent these limitations, authors have considered higher-moment methods that provide more fine-grained analyses of the cost distribution. Kura et al. [49] have shown that it is possible to encode higher-moments as monoids. We show how these monoids are $\mathcal{D}$ -algebras when equipped with the expected value algebra structure.

Definition 3.2 ([49, Definition 3.10]). For $n : \mathbb{N}$ we equip $(\mathbb{R}^+)^n$ with the following monoid structure: $(a_1, \dots, a_n) + (b_1, \dots, b_n) = (c_1, \dots, c_n)$, where $c_i = \sum_{0 \leq j \leq i} \binom{i}{j} a_j b_{i-j}$. Its algebra structure is given by $\mu : \mathcal{D}((\mathbb{R}^+)^n) \mapsto (\mathbb{E}(\mu_1), \dots, \mathbb{E}(\mu_n))$, where μ_i is the i -th marginal of μ .

Note that when $n = 1$, we recover the expected cost monad. When $n = 2$, the monoid structure specializes to $(x_1, y_1) + (x_2, y_2) = (x_1 + x_2, y_1 + y_2 + 2x_1x_2)$, which is analogous to the equation that holds whenever the random variables X and Y are independent:

$$\mathbb{E}((X + Y)^2) = \mathbb{E}(X^2) + 2\mathbb{E}(X)\mathbb{E}(Y) + \mathbb{E}(Y^2)$$

3.3 Weighted Programming and Weighted Cost

The concept of weighted programming has been explored before in the context of the quantitative semantics/linear logic literature [50] and has been recently proposed as a way of uniformly incorporating nondeterministic branching with the ability of weighting the current execution trace [11]. In this example, we are interested in reasoning about the weighted cost of weighted programs. First, we use a monad for finitely-supported weighted programming.

Definition 3.3. Given a semiring $\mathcal{R}$, we can equip the finitely supported functions $WA = A \rightarrow_{fs} \mathcal{R}$ with a monad structure.

Functor Assuming that $f : A \rightarrow B$ is a function, we define the functorial action of functions to be $F(f)(\varphi) = \lambda b. \sum_{a \in f^{-1}(b)} \varphi(a)$. The support of φ being finite is what makes the expression above be well-defined.

Unit The unit can be seen as a generalization of Dirac's delta distribution. Formally, $\eta(a)(a') = 1$ if $a = a'$ and 0 otherwise.

Kleisli extension The Kleisli extension can be seen as an integration operation. Assuming that $f : A \rightarrow W(B)$, we define $f^\#(\varphi) = \lambda b. \sum_{a:A} \varphi(a)f(a, b)$. The expression is well-defined by assumption that φ has finite support.

Some useful examples of such monads include the Boolean semiring $\{0, 1\}$ for nondeterminism, the natural numbers for counting-sensitive non-determinism and the interval $[0, \infty]$ for probability. Next we choose the following effectful notion of cost.

Definition 3.4. We can equip $\mathcal{R}$ with the following algebra structure $\varphi \mapsto \sum_{r \in \mathcal{R}} r\varphi(r)$.

In the finite probabilistic case we recover the expected cost monoid and in the boolean semiring case we can recover worst/best cost analyses as well by choosing different algebra structures.

3.4 Imprecise Probability and Worst-case Expected Cost

A well-known problem in the theory of programming languages is the non-existence of a distributive law between the probability and powerset monad [74]. That being said, recent advances have shown that there is a *weak* distributive law between those monads [25], which makes it possible to show that *convex* powersets can be equipped with a monad structure [56, Definition 8].

Lemma 3.5. *The convex powerset operation can be equipped with a monad structure $\mathcal{P}_c$.*

In such a programming context, it can be useful to assume a demonic approach to non-determinism, where we want to consider the worst-possible execution time. It is possible to combine this with the expected cost analysis and obtain the following monoid and $\mathcal{P}_c$ -algebra structure.

Definition 3.6. We can equip with positive real line $\mathbb{R}^+$ with the max expected value algebra structure, that is $C \mapsto \max_{\mu \in C} \mathbb{E}(\mu)$.

Therefore, the monad $\mathbb{R}^+ \times \mathcal{P}_c$ can be used to reason about the worst-case expected cost.

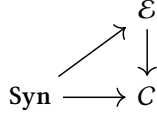


Fig. 5. Initial model

4 CATEGORICAL LOGICAL RELATIONS

In this section we put forth the necessary categorical machinery used in the soundness proofs of Section 5. Since we are interested in establishing soundness of infinitely many categorical models, we have to use more sophisticated notions of logical relations. Due to the technical nature of this section and the fact that we will use its results as a blackbox in the next section, readers who are mainly interested in compositional cost analysis may skip this section in a first reading.

Due to the relevance of logical relations as a proof technique in type theory and programming language theory, much work has been devoted to their mathematical study [8, 24, 30, 42, 44, 71]. By operating at a higher level of abstraction, it is possible to justify the soundness of logical relations models by judiciously applying well-known theorems from category theory. While such techniques are capable of constructing sophisticated logical relations [6, 34, 53, 70], existing techniques cannot scale to all the soundness properties we want to establish, which motivates the need for further development.

We begin by reviewing how proofs by categorical logical relations work. After going over some basic constructions and definitions from the categorical study of logical relations such as $\mathbf{CLat}_\wedge$ -fibrations, we introduce lax fibrations: a generalization of $\mathbf{CLat}_\wedge$ -fibrations that can also handle ordered aspects of logical relations. This plays an important role in justifying the soundness of our cost semantics with respect to the monad transformer cost semantics.

Next, we introduce *ordered* monads on categories enriched over partial orders and monotone functions, where the monad laws hold up to inequalities. After defining them, we prove Theorem 4.14, the main theorem of this section. It is a generalization of a theorem by Katsumata [42]: we show that ordered-monadic logical relations are stable under pullback, allowing us to build new logical relations models from old ones. We conclude by generalizing a result of Azevedo de Amorim et al. [8] that allows us to transfer CBV logical relations models to CBPV ones.

4.1 Categorical Logical Relations Proofs, in a Nutshell

The usual way of proving properties by logical relations is by inductively defining relations on types and then proving by induction the so-called fundamental theorem of logical relations. When approaching this from a category theory point of view, the proof is decomposed as depicted in Figure 5: the inductively-defined relation is replaced by a model of your calculus $\mathcal{E}$ equipped with a model morphism $\mathcal{E} \rightarrow C$; the inductive proof is replaced by initiality of the syntactic model $\mathbf{Syn}$, i.e. for every model C of your language, there is exactly one model morphism $\mathbf{Syn} \rightarrow C$. Under this setup, one can reason about the semantics in C by first reasoning about the semantics in the refined model $\mathcal{E}$ and then projecting it to C . Which is analogue to what happens in “analytic” approaches to logical relations, where the fundamental lemma shows that the category of types and relations is compatible with the type constructors and with the base semantics.

For more complicated calculi, it becomes harder to construct such refined models $\mathcal{E}$ by hand. Luckily, there is a robust theory that allows us to reuse older models to define new ones, even across

different families of calculi — we will exploit this in Lemma 4.19, which shows how to transform CBV logical relations models into CBPV ones.

4.2 Basic Definitions

One of the key observations from work in categorical logical relations is that many constructions from the practice of logical relations can be expressed through the language of fibrations [30]. Indeed, the specific cases of $\mathbf{CLat}_\wedge$ -fibrations capture the idea that logical relations should be thought of as “types with relations” and morphisms as relation-preserving morphisms. Before we formally define them, we need the following concept.

Definition 4.1. If $F : \mathcal{E} \rightarrow \mathcal{B}$ is a functor and $X : \mathcal{B}$, the fiber category $\mathcal{E}_X$ is the category where the objects are the preimage $F^{-1}(X)$ and the morphisms are the preimage $F^{-1}(id_X)$.

With this, we may now define $\mathbf{CLat}_\wedge$ -fibrations.

Definition 4.2. A $\mathbf{CLat}_\wedge$ -fibration is a functor $p : \mathcal{E} \rightarrow \mathcal{B}$ such that every fiber category $\mathcal{E}_X$ is a complete lattice, seen as a posetal category, and for every $\mathcal{B}$ morphism $f : X \rightarrow Y$ there is a reindexing functor $f^* : \mathcal{E}_Y \rightarrow \mathcal{E}_X$ that preserves $\wedge$.

There is an equivalent way of defining such fibrations as functors with certain morphism lifting properties. We will use such characterization when defining lax fibrations in Definition 4.9. Before, we show how many logical relations-flavored models of the simply typed λ -calculus can be presented as a $\mathbf{CLat}_\wedge$ -fibration, as we now illustrate.

Example 4.3 (Unary logical relations). Let $\mathbf{Pred}$ be the category where the objects are pairs $(A : \mathbf{Set}, X \subseteq A)$ and morphisms are functions $f : A \rightarrow B$ such that $f(X) \subseteq \mathcal{Y}$. The forgetful functor $\mathbf{Pred} \rightarrow \mathbf{Set}$ is a $\mathbf{CLat}_\wedge$ -fibration.

The advantage of formulating logical relations in this language is being able to use off-the-shelf theorems in order to create more sophisticated logical relations models out of simpler ones.

Example 4.4 (Binary logical relations). An important property of $\mathbf{CLat}_\wedge$ -fibrations is that they are stable under pullbacks. Therefore, it is possible to define binary logical relations using the following pullback, where $\Delta : \mathbf{Set} \rightarrow \mathbf{Set} \times \mathbf{Set}$ is the diagonal functor mapping a set A to $A \times A$.

$$\begin{array}{ccc} \mathbf{BinPred} & \longrightarrow & \mathbf{Pred} \times \mathbf{Pred} \\ \downarrow & \lrcorner & \downarrow \\ \mathbf{Set} & \xrightarrow{\Delta} & \mathbf{Set} \times \mathbf{Set} \end{array}$$

The category $\mathbf{BinPred}$ has pairs $(A, X \subseteq A \times A)$ as objects and morphisms $(A, X) \rightarrow (B, \mathcal{Y})$ are functions $f : A \rightarrow B$ such that $(f \times f)(X) \subseteq \mathcal{Y}$.

In practice, both of these categories are often used to handcraft logical relations. The abstract version of the “fundamental lemma of logical relations” is proving that categories such as $\mathbf{Pred}$ and $\mathbf{BinPred}$ are indeed models of the simply typed λ -calculus, i.e. they are Cartesian closed categories. While this approach works well, for more sophisticated languages, such as Moggi’s metalanguage, one has to work harder when defining the logical relations model. Luckily, it is also possible to accommodate these languages under the same categorical formalism. A fibered account of logical relations for monads is defined as follows:

Definition 4.5. Let $p : \mathcal{E} \rightarrow \mathcal{C}$ be a $\mathbf{CLat}_\wedge$ -fibration and $T : \mathcal{C} \rightarrow \mathcal{C}$ a (strong) monad. A p -monad lifting of T is a monad $\hat{T} : \mathcal{E} \rightarrow \mathcal{E}$ such that p strictly preserves the monad structure, i.e. it strictly preserves the bicartesian closed structure and $\hat{T}$ is a p -monad lifting of T .

We now define logical relations models for Moggi's metalanguage with positive inductive types.

Definition 4.6. A logical relations fibration for Moggi's metalanguage is a $\mathbf{CLat}_\wedge$ -fibration $p : (\mathcal{E}, \hat{T}) \rightarrow (\mathcal{B}, T)$ between bicartesian closed categories $\mathcal{E}$ and $\mathcal{B}$, paired with monads $\hat{T}$ and T , respectively, such that p strictly preserves the model structure. In our case, models are bicartesian closed categories with initial algebras for positive type constructors.

Under the formalism, defining a logical relation for a monad $T : C \rightarrow C$ corresponds to finding an appropriate logical relations fibration with codomain (C, T) . There is also a rich theory that has been developed in order to streamline this process. It is possible to show that such liftings satisfy the following pullback property:

Theorem 4.7 ([8, 42]). *Let $p : (\mathcal{F}, \hat{T}) \rightarrow (\mathcal{D}, T)$ be a logical relations fibration and $G : (C, S) \rightarrow (\mathcal{D}, T)$ a monad morphism. The following pullback exists in the category of strong monads and it is computed pointwise.*

$$\begin{array}{ccc} (\mathcal{E}, \hat{S}) & \longrightarrow & (\mathcal{F}, \hat{T}) \\ \downarrow & \lrcorner & \downarrow (p, =) \\ (C, S) & \xrightarrow{(F, \lambda)} & (\mathcal{D}, T) \end{array}$$

4.3 Lax Logical Relations Fibrations

In terms of logical relations, given a lifting of a monad T , the result above can be used to compute a canonical lifting of the monad S . Unfortunately, for many applications, the requirement that the (F, λ) leg in the pullback diagram above is a monad morphism is too strong. Intuitively, the monad morphism requirement enforces an airtight abstraction barrier between both monads. However, in practice abstractions may be contained but somewhat leaky. Therefore, there is some technical development required in order to address the shortcomings of existing techniques. Because these results are quite general, we believe that they are of broader interest beyond cost analysis.

We first introduce a new kind of fibration that interacts well with inequalities in the context of categories enriched over partial orders. Before we can use this new class of fibrations to construct logical relations models, we establish some of its basic theory and define useful examples. In the next section we use this theory to prove the soundness of **effcost**'s semantics with respect to the monad transformer semantics. We begin by defining **Poset**-enriched categories.

Definition 4.8. A **Poset**-enriched category C is a collection of objects C such that for every pair of objects $A, B : C$ there is a poset $C(A, B)$ with identity morphisms $id_A : C(A, A)$ for every object A and a monotonic composition operation $\circ : C(A, B) \times C(B, C) \rightarrow C(A, C)$.

These should be thought of as categories such that the morphisms can be equipped with a partial order structure which is respected by the composition operation. A **Poset**-functor between **Poset**-categories C and $\mathcal{D}$ is a functor $F : C \rightarrow \mathcal{D}$ such that its action $C(A, B) \rightarrow \mathcal{D}(FA, FB)$ is monotonic. Enriched categories are very common in categorical semantics. For example, enriching over complete partial orders (CPOs) is often used to give semantics to recursive programs [73].

The category **PosCat** contains categories enriched over partial orders as objects and their enriched functors as morphisms. We can now define lax fibrations.

Definition 4.9. Let $\mathcal{E}$ and C be two **Poset**-categories. A lax **Poset**-fibration is a **Poset**-functor $p : \mathcal{E} \rightarrow C$ such that:

- p is locally order reflective. That is, for every pair of arrows $f, g : X \rightarrow Y$, $p(f) \leq p(g)$ if, and only if, $f \leq g$.

- For every $f : A \rightarrow pY$ in $\mathcal{C}$ there is an X and $\hat{f} : X \rightarrow Y$ in $\mathcal{E}$ such that for every $\hat{h} : Z \rightarrow Y$ such that $p(\hat{h})$ factors through some $g : pZ \rightarrow A$, up to an inequality $\leq$, there is a unique $\hat{g} : Z \rightarrow X$ that factors $\hat{h}$ up to an inequality $\leq$. In terms of diagrams:

$$\begin{array}{ccc}
 Z & \xrightarrow{\hat{h}} & Y \\
 \text{---} \hat{g} \text{---} & \searrow \leq & \downarrow \hat{f} \\
 & X & \\
 & \text{---} \hat{f} \text{---} & Y
 \end{array}
 \quad
 \begin{array}{c}
 \mathcal{E} \\
 \downarrow p \\
 \mathcal{C}
 \end{array}$$

$$\begin{array}{ccc}
 pZ & \xrightarrow{p(\hat{h})} & pY \\
 \text{---} g \text{---} & \searrow \leq & \downarrow f \\
 & A & \\
 & \text{---} f \text{---} & pY
 \end{array}$$

This is similar to notions of fibered 2-categories [9, 15], though the exact connection to previous work is still not clear. Throughout the rest of the paper we will simply refer to these fibrations as lax fibrations. These should not be confused with the notion of lax logical relation defined by Plotkin et al. [63].

We now present the canonical example of a lax fibration. This example is the powerhouse behind our soundness proof in the next section and can be seen as a variant of the **Pred** category.

Example 4.10. Let **CPO** be the category of CPOs and Scott-continuous functions. The category **Pred**_{dc} has pairs $(X : \mathbf{CPO}, \mathcal{A} \subseteq X)$ as objects, where $\mathcal{A}$ is a downward-closed subset of X , and Scott continuous functions that preserve the predicate. The forgetful functor **Pred**_{dc} $\rightarrow$ **CPO** is a lax fibration and we now prove this claim.

Let $f : A \rightarrow B$ be a monotonic function and $\mathcal{Y} \subseteq B$ be a downwards closed predicate. The set $X = \{a : A \mid f(a) \in \mathcal{Y}\}$ is downwards closed, since if $a' \leq a$ and $a \in X$, then $f(a') \leq f(a)$ which implies $f(a') \in \mathcal{Y}$ by $\mathcal{Y}$ being downwards closed.

Furthermore, f lifts to a morphism between $(A, X) \rightarrow (B, \mathcal{Y})$ by definition of X . Next, assume that there is $h : (C, \mathcal{Z}) \rightarrow (B, \mathcal{Y})$ and $g : C \rightarrow A$ such that $f \circ g \leq h$. We want to show that g lifts to a morphism $(C, \mathcal{Z}) \rightarrow (A, X)$. Unfolding the definitions, assume that $c \in \mathcal{Z}$. We want to prove that $g(c) \in X$, i.e. $f(g(c)) \in \mathcal{Y}$, which follows from $h(c) \in \mathcal{Y}$, and the assumptions $f \circ g \leq h$ and $\mathcal{Y}$ being downwards closed.

4.4 Ordered monads and their lifting

In order to take full advantage of lax fibrations, we need to introduce ordered monads and their morphisms. As shown in previous work [7], monad morphisms are too strict a definition when one wants to use a logical relation for proving that one semantics bounds another one. Therefore, we need to consider weaker notions of morphisms that captures reasoning “up to an inequality”.

Definition 4.11. Let $\mathcal{C}$ be a **Poset**-category. A lax ordered monad is a triple (T, η, μ) , where $T : \mathcal{C} \rightarrow \mathcal{C}$ is a **Poset**-functor, $\eta : 1 \rightarrow T$ and $\mu : T^2 \rightarrow T$ are natural transformations such that the monad laws hold up to an inequality. In string diagrammatic notation:

A lax ordered monad is strict if the monad laws commute on the nose. Throughout this paper, we refer to strict ordered monads simply as ordered monads.

Definition 4.12. Let (S, η^S, μ^S) and (T, η^T, μ^T) be two lax ordered monads. An ordered monad morphism between S and T is a functor $F : \mathcal{C} \rightarrow \mathcal{D}$ and a natural transformation $\lambda : SF \rightarrow FT$ such that

$$\begin{array}{ccc}
 \begin{array}{c} F \\ \eta^S \\ F \end{array} & \leq & \begin{array}{c} F \\ \eta^T \\ F \end{array} \\
 \begin{array}{c} S \\ \mu^S \\ F \end{array} & \leq & \begin{array}{c} S \\ \mu^T \\ F \end{array}
 \end{array}$$

We say that an ordered monad morphism is strict if the inequalities commute exactly. We now show where ordered monad morphisms naturally appear.

Example 4.13 ([7]). Let $\mathcal{D}_{\leq 1}$ be the subprobability monad. While in the probabilistic case there is a monad morphism $\mathcal{D}(\mathbb{N} \times -) \rightarrow [0, \infty] \times \mathcal{D}$ that maps the first marginal $\mathcal{D}(\mathbb{N})$ to its expected value, such monad morphism does not extend to the subprobabilistic case. Instead, there is an ordered monad morphism between them, where the multiplication law holds up to an inequality.

In the next section, we show that this example is a manifestation of a more general phenomenon. Under some conditions, we can construct non-strict monad morphisms between such monads.

Logical Relations for Ordered Monads. An important result in categorical approaches to logical relations for monads shows that monad liftings themselves are fibered over monads [42]. Such results are used to, given a lifting of a monad T and a monad morphism $S \rightarrow T$, define a canonical lifting of S . We now prove a generalization of such a theorem to the ordered case. Let $\mathbf{OMon}$ be the category of lax ordered monads and ordered monad morphisms and let $\mathbf{OMon}_\ell$ be the category of lax ordered monad liftings and pairs of ordered monad morphisms making the appropriate diagrams commute.

Theorem 4.14 (cf. Appendix). *The codomain functor $\mathbf{OMnd}_\ell \rightarrow \mathbf{OMnd}$ is a fibration.*

By the pullback stability of fibrations, the main theorem of this section is a simple corollary.

Corollary 4.15. *Our logical relations fibration is given by the following pullback:*

$$\begin{array}{ccc}
 \mathcal{E} & \xrightarrow{\quad} & \mathbf{OMnd}_\ell \\
 q \downarrow & \lrcorner & \downarrow p \\
 \mathbf{Mnd} & \hookrightarrow & \mathbf{OMnd}
 \end{array}$$

That is, the category $\mathcal{E}$ is the category of strict ordered monads liftings and possibly lax monad morphisms. The corollary above means that given an monad lifting and a lax monad morphism, then there is a canonical monad lifting on the pullback, much like in Theorem 4.7. We may now define our notion of lax logical relation.

Definition 4.16. A lax logical relation fibration is logical relation fibration $p : \mathcal{E} \rightarrow \mathcal{B}$ such that it is also a lax fibration.

Using this result, it is possible to use the fibered structure of the codomain fibration and liftings of monads along ordered monad morphisms. This generalizes the unordered version of this theorem and will be crucial when proving the soundness of our denotational semantics.

4.5 From CBV to CBPV

The corollary above can be used to define new logical relation models from old ones. However, the monad-centric approach advocated for in Corollary 4.15 does not perfectly match up the categorical structure of CBPV, only that of CBV. Luckily, much like from every CBV model it is possible to construct a canonical CBPV model, the same is true about their logical relations counterparts. In order to simplify the presentation, we restrict ourselves to defining CBPV logical relations for its algebraic semantics:

Definition 4.17 ([8]). An algebraic (lax) CBPV logical relation is a pair of (lax) $\mathbf{CLat}_\wedge$ -fibrations $p : \mathcal{E} \rightarrow C$ and $q : \mathcal{E}^{\hat{T}} \rightarrow C^T$ such that the adjunction $\mathcal{E} \dashv \mathcal{E}^{\hat{T}}$ is a lifting of the adjunction $C \dashv C^T$.

We now prove that every lax logical relations model gives rise to a canonical lax CBPV fibration. First we need the following lemma.

Lemma 4.18. *If $p : (\mathcal{E}, \hat{T}) \rightarrow (C, T)$ is a lax logical relation fibration then its restriction to the category of algebras $p : \mathcal{E}^{\hat{T}} \rightarrow C^T$ is a lax fibration.*

PROOF. The proof is analogue to the fibered case, where we use the universal property of Cartesian liftings to define the appropriate algebra structure. $\square$

We can now prove the following lemma:

Lemma 4.19 (cf. Azevedo de Amorim et al. [8, Lemma 4.18]). *If p is a CBV logical relations model, then (p, p) is a CBPV logical relations model.*

Therefore, while the definition of CBPV models is more involved than its CBV counterparts, the theorem above shows that in order to define a logical relation for CBPV, it suffices to construct a CBV one and transfer it through the lemma above.

5 GENERAL COST SOUNDNESS THEOREMS

In this section we prove soundness properties of our effectful cost semantics with respect to two different cost semantics that have been considered before in the literature. As mentioned in the previous section, the proofs follow by logical relations arguments. For the monad transformer semantics we require the generality afforded to us by lax fibrations.

We start by comparing our semantics to continuation-passing-style approaches to cost analysis. We state our soundness theorem by reducing it to an instance of the effect simulation problem [43] [22, Chapter 3] which has been elegantly formalized in the setting of categorical logical relations as gluing monads along monad morphisms. Something that helps our analysis is the fact that the continuation monad is somewhat of a universal monad: every monadic effect can be implemented by a continuation monad.

Next, we compare our semantics to the writer monad transformer approach to effectful cost analysis advocated for by Grodin et al. [26] with the **decalf** type theory. Compared to the continuation semantics, its soundness proof makes use of the technology of lax fibrations developed in the previous section and Corollary 4.15, going beyond the scope of existing techniques.

Continuation-Passing-Style cost analysis. Kaminski et al. [38] initiated a research programme of using CPS semantics for reasoning about effectful notions of cost. Such techniques have been applied to amortized expected runtime [12], quantum programming [4], etc., and given a recent categorical presentation by Kura [48], through the use of continuation monads. While versatile, using such a large monad obfuscates natural reasoning principles from your base effect. This was illustrated recently by Azevedo de Amorim [7], where one of his semantics validated useful and

sensible program equations such as commutativity, while the continuation semantics can essentially never satisfy such equations [16, Example 30].

In order to relate this semantics to **effcost**'s one, we will use the fact that, by choosing the response type $\mathbb{A}$ for your continuation monad, there will be a canonical monad morphism $\mathbb{A} \times T \rightarrow K_{\mathbb{A}}$, where $K_{\mathbb{A}} = (- \rightarrow \mathbb{A}) \rightarrow \mathbb{A}$ is the continuation monad with response type $\mathbb{A}$. Then, we use this monad morphism to formally relate both semantics, as stated in Theorem 5.1.

Monad Transformer Semantics. Grodin et al. [26] show that by using the canonical writer monad transformer $T(\mathbb{C} \times -)$, given a monoid object $\mathbb{C}$ and a strong monad T , it is possible to reason about cost in effectful programs. However, as noted by Azevedo de Amorim [7], depending on the base effect, such analyses may validate undesirable program equations which are not validated by the **effcost** semantics, as illustrated by Example 2.8. Furthermore, such a semantics is not compositional with respect to many analyses, e.g. in order to compute the expected cost of a program, one must first compositionally compute the cost distribution $\mathcal{D}(\mathbb{N})$ and then non-compositionally compute its expected value.

Unfortunately, it is not so clear how to relate this semantics to the one presented in this work. While the continuation semantics has the benefit of being the target of a canonical monad morphism, this is not the case for the monad transformer semantics. Fortunately, in many cases it is possible to construct ordered monad morphisms, allowing us to bound one cost semantics with respect to the other one. In Section 5.2, we study how properties on the base monad and cost analysis can translate to different soundness theorems.

Throughout this section we assume that our categories are locally small.

5.1 Continuation-Passing-Style Soundness

An important property of continuation monads is that they are, in a certain way, the largest monad there are: for every (strong) monad T and T -algebra (A, α) there is a canonical monad morphism from T into K_A . This makes these monads quite flexible and, by choosing an appropriate answer type, can give semantics to any monadic effect.

In the probabilistic case, Kaminski et al. [38] choose $[0, \infty]$ as their answer type, obtaining the continuation monad $(- \rightarrow [0, \infty]) \rightarrow [0, \infty]$, often referred to as the pre-expectation monad. With this monad, it is possible to interpret probability distributions through their “integrator” semantics, e.g. the Bernoulli distribution is interpreted as the pre-expectation $\lambda f. \frac{1}{2}f(0) + \frac{1}{2}f(1)$ and, more generally, a distribution over the natural numbers μ is interpreted as $\lambda f. \sum_{n \in \mathbb{N}} \mu(n)f(n)$. Under this semantics, the cost operation is interpreted as $\lambda f x. 1 + f x$. One of the limitations of this semantics can be observed in the following example.

In order to justify the soundness of our cost semantics we will prove a vast generalization of the results developed by Avanzini et al. [5, Section 7] who showed that the pre-expectation semantics can be factored as a cost component and a probability component: we show that under appropriate circumstances, the CPS semantics can be factored as a cost component plus a “CPS” component. The proof follows by reducing our problem to the effect simulation problem. We now formally state the soundness property.

Theorem 5.1. *Assuming that $\llbracket - \rrbracket_{\text{effcost}}$ is the denotational semantics described in Section 2.1 and that $\llbracket - \rrbracket_{\text{cps}}$ is the algebraic semantics of CBPV given by the continuation monad $K_{\mathbb{A}}$, for every program $\cdot \vdash_c t : F\tau$, $\pi_1(\llbracket t \rrbracket_{\text{effcost}}) = \llbracket t \rrbracket_{\text{cps}}(0)$, where $\pi_i : A_1 \times A_2 \rightarrow A_i$ is the i -th projection morphism.*

That is, the costful component of **effcost**'s semantics is equal to the costful component of the continuation semantics. The proof of this theorem can be found at the end of this subsection.

Effect Simulation Property. The property above can be understood as an instance of the effect simulation property. This property establishes when two different implementations of the same effect can be formally related. The canonical example is non-determinism, which can be implemented using either the list monad $[-]$ or the powerset monad $\mathcal{P}$.

Using the fact that the monads $[-]$ and $\mathcal{P}$ gives rise to their own CBPV semantic interpretation $\llbracket \cdot \rrbracket_{[-]}$ and $\llbracket \cdot \rrbracket_{\mathcal{P}}$, respectively, and the monad morphism $\text{elem} : [-] \rightarrow \mathcal{P}$ that maps each list to its set of elements, the effect simulation property can be stated as follows.

Theorem 5.2 (simplified effect simulation property). *For every non-deterministic program $\cdot \vdash t : \mathbb{N}$, $\text{elem}(\llbracket t \rrbracket_{[-]}) = \llbracket t \rrbracket_{\mathcal{P}}$, where $\llbracket \cdot \rrbracket_{[-]}$ and $\llbracket \cdot \rrbracket_{\mathcal{P}}$ are the algebraic CBPV semantics given by the list and powerset monads, respectively.*

More generally, under certain circumstances, monad morphisms $S \rightarrow T$ may be used to prove an effect simulation property between the monadic semantics given by S and T [43]. The following example is helpful.

Example 5.3. If T is a strong monad on a Cartesian closed category $\mathcal{C}$, for every T -algebra $\alpha : TA \rightarrow A$ there is a canonical monad morphism $T \rightarrow K_A$ defined as follows $TX \times (X \rightarrow A) \xrightarrow{\sigma} T(X \times (X \rightarrow A)) \xrightarrow{T(\text{ev})} T(A) \xrightarrow{\alpha} A$. Notice the similarity between this morphism and the algebra structure for arrow types from Definition 2.4.

Therefore, in our case, we want to find an appropriate algebra for the effectful cost monad $\mathbb{A} \times T$ —and use the canonical monad morphism into the continuation monad. This has a simple solution:

Lemma 5.4 (cf. Appendix). *Every monoid $\mathbb{A}$ in $\mathcal{C}^T$ can be equipped with a $\mathbb{A} \times T$ —algebra structure.*

PROOF. We equip $\mathbb{A}$ with the following algebra structure $\mathbb{A} \times T\mathbb{A} \xrightarrow{\mathbb{A} \times \alpha} \mathbb{A} \times \mathbb{A} \xrightarrow{+} \mathbb{A}$. The proof that this satisfies the algebra axioms can be found in the appendix. $\square$

Therefore, there is a canonical monad morphism $\gamma : \mathbb{A} \times T \rightarrow K_{\mathbb{A}}$. By unfolding the construction of γ , it follows that $\gamma_{\mathbb{A}}(r, \mu) = r + \gamma'_{\mathbb{A}}(\mu)$, where γ' is the canonical monad morphism $T \rightarrow K_{\mathbb{A}}$ which exists by assumption that $\mathbb{A}$ is a T -algebra.

Proof of the soundness theorem. Here is the road map of the proof:

- We construct a logical relation fibration $\mathbf{BinRel} \rightarrow \mathcal{C} \times \mathcal{C}$, for some category of binary relations $\mathbf{BinRel}$.
- We define an appropriate lifting of the monad $(\mathbb{A} \times T) \times K_{\mathbb{A}} : \mathcal{C} \times \mathcal{C} \rightarrow \mathcal{C} \times \mathcal{C}$.
- We use Lemma 4.19 to define the CBPV logical relation and conclude.

We now define the category $\mathbf{BinRel}$.

Definition 5.5. The category $\mathbf{BinRel}(\mathcal{C})$ has pairs $(A, B, \mathcal{X} \subseteq \mathcal{C}(1, A) \times \mathcal{C}(1, B))$ as objects and morphisms are $\mathcal{C}$ morphisms $f : A \rightarrow C$ and $g : B \rightarrow D$ such that $(\mathcal{C}(1, f) \times \mathcal{C}(1, g))(\mathcal{X}) \subseteq \mathcal{Y}$.

This category has been used for doing relational reasoning between different semantics [43, 46]. Such analyses require the following lemma.

Lemma 5.6. *The forgetful functor $\mathbf{BinRel}(\mathcal{C}) \rightarrow \mathcal{C} \times \mathcal{C}$ is a logical relation fibration.*

PROOF. This category can be defined as the following pullback along a Cartesian morphism.

$$\begin{array}{ccc} \mathbf{BinRel} & \longrightarrow & \mathbf{Pred} \\ \downarrow & \lrcorner & \downarrow \\ \mathcal{C} \times \mathcal{C} & \xrightarrow{\times} & \mathcal{C} \end{array}$$

□

In order to get the right soundness property out of our logical relations, we need a final insight. We want to show that the elements of T that “live inside” $K_{\mathbb{A}}$ satisfy the property that whenever the input continuation is the constant zero one, the output is zero. We formalize this by defining the following equalizer.

$$K_{\mathbb{A}}^0 \xrightarrow[\varphi]{\hookrightarrow} K_{\mathbb{A}} \xrightarrow[0]{ev_0} \mathbb{A}$$

Intuitively, $K_{\mathbb{A}}^0$ is the subset of $K_{\mathbb{A}}$ such that whenever you evaluate them at the constant zero continuation (the upper morphism ev_0), they output the constant 0 (the lower morphism 0). We can use the universal property of equalizers to prove the following fact.

Lemma 5.7. *The morphism $\gamma' : T \rightarrow K_{\mathbb{A}}$ factors through $K_{\mathbb{A}}^0$, i.e. for every $\mu : TA, \gamma'_A(\mu)(\lambda x. 0) = 0$.*

The final step in setting up our logical relation consists of finding an appropriate lifting of the monad $K_{\mathbb{A}} \times K_{\mathbb{A}}$. Luckily, since we assume that our logical relation fibration strictly preserves the Cartesian closed structure, in order to lift $K_{\mathbb{A}}$, it suffices to lift $\mathbb{A}$, which we now define.

Definition 5.8. The lifted cost monoid is $(\mathbb{A}, \mathbb{A}, =_{\mathbb{A}})$, where $=_{\mathbb{A}}$ is the equality relation on $C(1, \mathbb{A})$.

Now we can finally define our logical relation fibration as the pullback of the following diagram.

$$\begin{array}{ccc} & (\mathbf{BinRel}(C), \hat{K}\mathbb{A}) & \\ & \downarrow & \\ (C \times C, (\mathbb{A} \times T) \times K_{\mathbb{A}}) & \longrightarrow & (C \times C, (K_{\mathbb{A}}) \times K_{\mathbb{A}}) \end{array}$$

When we unfold the construction of the lifted monad, we recover the following monad on $\mathbf{BinRel}(C)$, it maps triples (X, Y, X) to $(\mathbb{A} \times T(X), K_{\mathbb{A}}(Y), \tilde{X})$, where $\tilde{X} = \{(f, g) \mid (\gamma \circ f, g) \in \hat{K}_{\mathbb{A}}(X)\}$ and $\hat{K}_{\mathbb{A}}(X)$ uses the familiar “logical relations” implication: they are the pairs $(F, G) : K_{\mathbb{A}}(X) \times K_{\mathbb{A}}(Y)$ such that if its input continuations $(f, g) : (X \rightarrow \mathbb{A}) \times (Y \rightarrow \mathbb{A})$ satisfy the arrow type logical relations then $F(f) = G(g)$. We now prove our continuation semantics soundness theorem.

PROOF OF THEOREM 5.1. When we compute the pullback above, we will get a logical relations fibration over the monad $(\mathbb{A} \times T) \times K_{\mathbb{A}}$ on $\mathbf{BinRel}$. We then apply Lemma 4.19 to get a CBPV logical relations model. Under this model, the effectful operations are interpreted as follows:

$$\begin{aligned} \llbracket \text{op} \rrbracket &= (\llbracket \text{op} \rrbracket_{\text{effcost}}, \gamma \circ \llbracket \text{op} \rrbracket_{\text{effcost}}) \\ \llbracket \text{charge } V \rrbracket &= (\llbracket V \rrbracket_{\text{effcost}} \times \eta, \lambda f x. \llbracket V \rrbracket_{\text{cps}} + f x) \end{aligned}$$

Note that by forcing the continuation component of $\llbracket \text{op} \rrbracket$ to be $\gamma \circ \llbracket \text{op} \rrbracket_{\text{effcost}}$ we are making our logical relations model correct by construction.

Now, let $\cdot \vdash t : F\tau$. Interpreting it in the logical relations model above and by using the initiality of syntax we get that $\pi_1(\llbracket t \rrbracket_{\text{effcost}}) + \gamma'(\pi_2(\llbracket t \rrbracket_{\text{effcost}}))(f) = \llbracket t \rrbracket_{\text{cps}}(g)$ for every pair of continuations $(f, g) : (\llbracket \tau \rrbracket_{\text{effcost}} \rightarrow \mathbb{A}) \times (\llbracket \tau \rrbracket_{\text{cps}} \rightarrow \mathbb{A})$ satisfying the appropriate logical relations condition. By construction, the pair $(\lambda x. 0, \lambda x. 0)$ does satisfy this condition.

We now conclude: $\pi_1(\llbracket t \rrbracket) + \gamma'(\pi_2(\llbracket t \rrbracket))(\lambda x. 0) = \llbracket t \rrbracket_{\text{cps}}(\lambda x. 0)$, which by Lemma 5.7, implies $\gamma'(\pi_2(\llbracket t \rrbracket))(0) = 0$ and therefore, $\pi_1(\llbracket t \rrbracket) = \llbracket t \rrbracket_{\text{cps}}(0)$. Once again, the equality comes from the fact that we chose $(\mathbb{A}, \mathbb{A}, =)$ as our lifting for the effectful monoid. □

5.2 Monad Transformer Soundness

Now, we prove soundness properties with respect to the monad transformer semantics presented in Section 2.2. In order to understand what makes this soundness proof different from the previous one, consider Example 4.13. The monad transformer cost semantics interprets probabilistic programs as subdistributions $\mu : \mathcal{D}_{\leq 1}(\mathbb{N} \times \tau)$ over costs and the output type. On the other hand, our cost semantics interprets such a program as a pair $(r, \nu) : [0, \infty] \times \mathcal{D}_{\leq 1}(\tau)$, where r is the expected cost.

In this case, the appropriate soundness property is the expected value of the first marginal μ_1 being equal to r . Unfortunately, this property does not hold. A simple counterexample is the program `charge 1;  $\perp$` . In this case, the monad transformer semantics says that the expected value of running this program should be 0, while our semantics says that it is 1, much like Example 2.8.

Fortunately, it is still possible to show that our cost semantics bounds the expected value of the cost distribution. This is where we need to make use of ordered monad morphisms. This makes the soundness theorem less straightforward to state and prove.

In this section we set up necessary conditions on monads and monoids that imply cost soundness properties. At a technical level, given monoid objects $\mathbb{C}$ and $\mathbb{A}$, we want to understand when we can find ordered monad morphisms $T(\mathbb{C} \times -) \rightarrow \mathbb{A} \times T$, strict or otherwise. The strictness of the morphism is what dictates whether the cost soundness theorem is a tight bound or not.

Different cost monoids. There are situations where the cost analysis happens between different monoids. For example, in the higher-moments example presented in Section 3, the n -th order moment cost analysis should be proved sound with respect to a map $\mathcal{D}(\mathbb{N}) \rightarrow (\mathbb{R}^+)^n$. In the case of effectful monoids, the algebra structure $TM \rightarrow M$ is a canonical choice for the cost analysis.

Definition 5.9. Let $T : C \rightarrow C$ be a monad and $(A, \alpha : TA \rightarrow A, 0, +)$ a monoid in the categories of algebras C^T . The monoid is said to be strongly effectful if it makes the following diagram commute:

$$\begin{array}{ccc} T(A \times T1) & \xrightarrow{T(st)} T^2(A \times 1) & \xrightarrow{\mu} T(A \times 1) \\ & \searrow T(\pi_1) & \downarrow T(\pi_1) \\ & & T(A) \end{array}$$

The property above requires the algebra structure on the monoid to interact well with the monad structure. The reason we care about this property is that it implies the following property.

Lemma 5.10 (cf. Appendix). *Let $T : C \rightarrow C$ be a strong monad. If the monoid $\mathbb{A}$ is strongly effectful then $\langle \alpha \circ T\pi_1, T\pi_2 \rangle : T(\mathbb{A} \times -) \rightarrow \mathbb{A} \times T$ is a (strict) ordered monad morphism.*

Tight bounds. We begin by considering the cases where we can prove that **effcost**'s semantics is a sound approximation to the monad transformer semantics. We start by stating the theorem we want to prove. Due to the similarity between this theorem and Theorem 5.1, their proofs are basically the same.

Theorem 5.11 (Tight soundness property). *If there is a monad morphism $\varphi : T(\mathbb{C} \times -) \rightarrow \mathbb{A} \times T-$, there is an interpretation of **effcost** such that for every $\cdot \vdash_c t : F\tau$, $\pi_1(\llbracket t \rrbracket_{\text{effcost}}) = k(\llbracket t \rrbracket_{MT})$, where $\llbracket - \rrbracket_{MT}$ is the monad transformer CBPV semantics and $k : T(\mathbb{C}) \rightarrow \mathbb{A}$ is $\pi_1 \circ \gamma$, that is, it is the cost analysis generated by the monad morphism φ .*

PROOF. The proof is basically the same as Theorem 5.1. We start with the monad lifting of $\mathbb{A} \times T$ calculated for the CPS soundness proof and once again we pullback along a monad morphism, but now it is along $\varphi \times id$. Unfolding the definitions give us our soundness theorem.

It is important to highlight the fact that if $\mathbb{A} \neq \mathbb{C}$ then we have to be a bit careful of which operations are in the metalanguage. For example, the charge operation has to be parametrized by elements of $\mathbb{C}$ and not $\mathbb{A}$. The same will be true for the approximate soundness property. $\square$

Unfortunately, contrary to the CPS example, it is not always the case that such monad morphism exists. In order to better navigate the space of possibilities, we now show a couple of sufficient conditions that guarantee the existence of the monad morphism mentioned above.

Example 5.12 (Affine monads). A monad $T : \mathcal{C} \rightarrow \mathcal{C}$ on a Cartesian category $\mathcal{C}$ is said to be affine if the unit $\eta_1 : 1 \rightarrow T1$ is an isomorphism and there is a natural transformation $\nabla : T - \times T \Rightarrow T(- \times =)$ making certain diagrams commute. Every monoid in $\mathcal{C}^T$ is strongly effectful. This follows from the following commutative diagram:

$$\begin{array}{ccccc}
 T(A \times 1) & \xrightarrow{T(A \times \eta)} & T(A \times T1) & \xrightarrow{T(st)} & T^2(A \times 1) \\
 & \searrow \eta & & & \downarrow \mu \\
 & & & & T(A \times 1) \\
 \downarrow T(\pi_1) & & & & \downarrow T(\pi_1) \\
 T(A) & \xlongequal{\quad\quad\quad} & & & T(A)
 \end{array}$$

The lower diagram by one of the monad laws and the upper one by one of the strengths. This diagram commuting implies that the strongly effectful one commutes by the assumption that T is affine and, therefore, $T(A \times \eta_1)$ is an isomorphism.

While this example already encompasses some useful examples, we show that we can go beyond affine monads. If we pay close attention to Example 4.13, we see that what breaks the tight soundness property is the possibility of losing mass in the execution of possibly non-terminating programs, which leads to a distortion of the expected value.

Example 5.13 (Other examples). Let $\mathcal{P}_{ne}$ be the non-empty powerset monad. Every monoid in its category of algebras is strongly effectful, as a direct calculation shows.

Upper bound. Unfortunately, in general there are many monoids that are not strongly effectful, meaning that we cannot use the theory behind the effect simulation literature in order to establish soundness. We now propose a relaxation of strongly effectful monoids that allows us to construct a lax monad morphism and, therefore, establish a relaxation of the effect simulation property.

Definition 5.14. An effectful cost analysis is a lax monoid morphism between a monoid $\mathbb{C}$ and an effectful monoid $\mathbb{A}$, that is, it is a map $f : T\mathbb{C} \rightarrow \mathbb{A}$ such that

$$\begin{array}{ccc}
 T\mathbb{C} \times T\mathbb{C} & \xrightarrow{f \times f} & \mathbb{A} \times \mathbb{A} \\
 \downarrow +_{\mathbb{C}} & \leq & \downarrow +_{\mathbb{A}} \\
 T\mathbb{C} & \xrightarrow{f} & \mathbb{A}
 \end{array}
 \qquad
 \begin{array}{ccc}
 1 & \searrow 0_{\mathbb{A}} & \\
 \eta \circ 0_{\mathbb{C}} \downarrow & \leq & \\
 T\mathbb{C} & \xrightarrow{f} & \mathbb{A}
 \end{array}$$

Lemma 5.15. Every lax monoid morphism induces an ordered monad morphism $T(\mathbb{C} \times -) \rightarrow \mathbb{A} \times T-$. Furthermore, if the monoid morphism is strict, so is the ordered monad morphism.

While establishing sufficient conditions for such a property is harder than in the strict case, we now show how this definition encompasses useful examples.

Example 5.16. Consider the subprobability distribution monad $\mathcal{D}_{\leq 1}$ and equip $[0, \infty]$ with the expected value algebra structure. In this case, a direct calculation shows that the inequality holds.

Example 5.17. An example of a non-commutative monad that satisfies this property is the list monad. The intuition is that this property is capturing something about when effects can have an “error state”. In this case, the empty list is a kind of “error state”.

We can now state our effect approximation soundness property.

Theorem 5.18. *If there is an ordered monad morphism $\psi : T(\mathbb{C} \times -) \rightarrow \mathbb{A} \times T-$, there is an interpretation of **effcost** such that for every $\vdash_c t : F\tau$, $\llbracket t \rrbracket \leq \psi(\llbracket t \rrbracket)$.*

The proof of this theorem is more complicated and requires a different logical relations model, which we now define. It is very similar to the effect simulation problem, except that we have to be a bit more careful when dealing with the asymmetry surfaced by the inequality we are trying to prove. Much like in the effect simulation problem, we define our relational model by a pullback into a unary logical relation model.

Lemma 5.19 (cf. Appendix). *The lax fibration $U : \mathbf{Pred}_{dc} \rightarrow \mathcal{C}$ is a logical relations lax fibration.*

Once again, it is possible to define the binary logical relation fibration by a pullback. However, since we are trying to prove an inequality, we have to be a bit careful when choosing what category to be the base of the fibration.

Definition 5.20. We now define our relational lax logical relation by the following pullback

$$\begin{array}{ccc} \mathbf{BinPred}_{dc} & \longrightarrow & \mathbf{Pred}_{dc} \\ \downarrow & \lrcorner & \downarrow \\ \mathcal{C} \times |C| & \xrightarrow{\times} & \mathcal{C} \end{array}$$

where $|C|$ is the discretization of C , i.e. we collapse the posets $C(X, Y)$ to the discrete partial order $f \leq g$ if, and only if, $f = g$. By the bifunctor $\times$ being Cartesian, Lemma 5.19 and lax fibrations being stable under pullback (cf. Theorem B.2 in the Appendix), $\mathbf{BinPred}_{dc} \rightarrow \mathcal{C} \times |C|$ is a lax logical relations fibration. Concretely, the objects of $\mathbf{BinPred}_{dc}$ are triples $(A, B, \mathcal{X} \subseteq C(1, A) \times C(1, B))$ such that $\mathcal{X}$ is only downwards closed on A .

We can prove the lax soundness property.

PROOF OF THEOREM 5.18. As we have seen in the definition above, the category $\mathbf{BinPred}_{dc}$ and its forgetful functor $\mathbf{BinPred}_{dc} \rightarrow \mathcal{C} \times |C|$ is a lax logical relation fibration.

In order to construct the CBPV logical relations model, we will define a lift of $(T(\mathbb{C} \times -)) \times (\mathbb{A} \times T)$ and then apply Lemma 4.19 and conclude the proof by inspection of the logical relations model.

The lift of the monad is given by first lifting $(\mathbb{A} \times T) \times (\mathbb{A} \times T)$ and then pulling the lifting back along the ordered monad morphism $(T(\mathbb{C} \times -)) \times (\mathbb{A} \times T) \xrightarrow{\psi \times id} (\mathbb{A} \times T) \times (\mathbb{A} \times T)$, which exists by hypothesis. Because the fibration is assumed to preserve products, it suffices to lift $\mathbb{A} \times \mathbb{A}$ and $T \times T$. The cost part of the monad is given by $(\mathbb{A}, \mathbb{A}, \leq_{\mathbb{A}})$, while we lift $T \times T$ as done in the effect simulation example: by pulling back along the canonical monad morphism $T \rightarrow K_{\mathbb{A}}$.

The proof that this proposed lift is indeed a monad in $\mathbf{BinPred}_{dc} \times \mathbf{BinPred}_{dc}$ follows from Theorem 2.7, i.e. $(\mathbb{A}, \mathbb{A}, \leq_{\mathbb{A}})$ is an effectful monoid for the lifted monad of $T \times T$. $\square$

6 PARALLELISM AND PARALLEL COST IN **effcost**

We conclude the technical development of this paper by proposing an extension of **effcost** so that it can also accommodate parallel computation. We model this by specializing concurrent pseudomonads [62, 67], to our semantic context.

Definition 6.1 ([67, §3.2]). Let $(T, \eta, \mu) : C \rightarrow C$ be a strict strong ordered monad. We say that T is a concurrent ordered monad if there is a morphism $\psi : 1 \rightarrow T1$ and a natural transformation $\nabla_{A,B} : T(A) \times T(B) \rightarrow T(A \times B)$ such that (T, ∇, ψ) is a lax monoidal functor [14, Definition 6.4.1] and the following inequalities hold:

$$\begin{array}{ccc}
 \begin{array}{c} 1 \\ \eta_1 \left(\begin{array}{c} \leq \end{array} \right) \psi \\ T1 \end{array} & & \begin{array}{c} 1 \\ \psi \downarrow \\ T1 \end{array} \xrightarrow{\leq} \begin{array}{c} T^2 1 \\ \xrightarrow{T\psi} \end{array} \xrightarrow{\mu_1} T1
 \end{array}$$

$$\begin{array}{ccc}
 \begin{array}{c} X \times Y \xrightarrow{\eta_X \times \eta_Y} TX \times TY \\ \searrow \eta_{X \times Y} \quad \downarrow \nabla_{X,Y} \\ T(X \times Y) \end{array} & & \begin{array}{c} T^2 X \times T^2 Y \xrightarrow{\mu_X \times \mu_Y} TX \times TY \\ \nabla_{TX, TY} \downarrow \quad \searrow \nabla_{X,Y} \\ T(TX \times TY) \xrightarrow{T\nabla_{X,Y}} T^2(X \times Y) \xrightarrow{\mu_{X \times Y}} T(X \times Y) \end{array}
 \end{array}$$

From a semantics point of view, this is almost the same thing as a commutative monad, but the operation that runs programs in parallel must obey a few inequalities. Such inequalities are inspired from the work done on concurrent Kleene algebra [31, 41]. From a language design point of view, we accommodate the concurrent structure by adding the following typing rule to **effcost**:

$$\frac{\Gamma \vdash_c t_1 : F(\tau_1) \quad \Gamma \vdash_c t_2 : F(\tau_2) \quad \Gamma, x_1 : \tau_1, x_2 : \tau_2 \vdash_c u : \bar{\tau}}{\Gamma \vdash_c (x_1 || x_2) \leftarrow (t_1 || t_2); u : \bar{\tau}}$$

From an operational point of view, this construct runs t_1 and t_2 in parallel, binds the results to x_1 and x_2 , respectively, and continues as u .

In the algebraic semantics of CBPV, we interpret it as follows:

$$\Gamma \xrightarrow{\langle id, t_1, t_2 \rangle} \Gamma \times UF\tau_1 \times UF\tau_2 \xrightarrow{\Gamma \times \nabla} \Gamma \times UF(\tau_1 \times \tau_2) \xrightarrow{\sigma} UF(\Gamma \times \tau_1 \times \tau_2) \xrightarrow{UFu} UFU\bar{\tau} \xrightarrow{\alpha} U\bar{\tau}$$

The semantics is similar to the sequencing operation, with the exception that we have to apply the concurrent operation ∇ with the execution of t_1 and t_2 . While not every base monad can be equipped with such a structure, we now give a sufficient condition that allows us to define a concurrent cost monad. We now present a slight generalization of the definition of concurrent monoid introduced by Hoare et al. [31].

Definition 6.2. A concurrent monoid in a **Poset**-category is an object M equipped with two monoid structures $(M, 0, \oplus)$ and $(M, 1, \odot)$ such that $0 \leq 1$, $1 \oplus 1 \leq 1$, $0 \leq 0 \odot 0$ and it satisfies a weak exchange law $(a \oplus b) \odot (c \oplus d) \leq (a \odot c) \oplus (b \odot d)$.

Theorem 6.3. Assume that your base monad T is a concurrent ordered monad and that $\mathbb{A}$ is a concurrent monoid. The monad $\mathbb{A} \times T$ can be equipped with a canonical concurrent ordered monad structure.

PROOF. By rearranging the product, using the duoidal structure of $\mathbb{A}$ and commutative structure of T , we can define the concurrent structure as $\mathbb{A} \times TA \times \mathbb{A} \times TB \rightarrow \mathbb{A} \times \mathbb{A} \times TA \times TB \xrightarrow{\odot \times \nabla} \mathbb{A} \times T(A \times B)$. The proof that this is indeed a concurrent ordered monad follows from the fact that the product order is given componentwise and that T is assumed to be concurrent. $\square$

We now show how this structure is present in useful examples by showing how it can handle the combination of choice non-determinism, parallelism and worst-case cost.

Example 6.4. As an example, consider the Hoare powerdomain $\mathfrak{P} : \text{CPO} \rightarrow \text{CPO}$ which maps a CPO D to the set of Scott-closed subsets of D , ordered by inclusion. This commutative monad can be used to model angelic non-determinism [23, Chapter IV-8].

We now choose the extended additive natural numbers $\bar{\mathbb{N}}$, with the usual order, as our effectful cost monoids. The algebra structure we choose is $\max$. The other monoid structure on $\bar{\mathbb{N}}$ is $(\max, 0)$. This is often known as the max tropical semiring.

By the lemma above, the monad $\bar{\mathbb{N}} \times \mathfrak{P}$ can be equipped with a concurrent monad structure given by the concurrent monoid structure of $(\bar{\mathbb{N}}, +, 0, \max, 0)$. In this case, this monad captures the worst case cost of non-deterministic programs that may be run in parallel.

7 RELATED WORK

Type-based analyses. There is a vast literature on using type systems for doing cost analysis. The notable line of work on automatic amortized resource analysis (AARA) is behind a bunch of great work [27, 33, 54, 75] [32, 36, 59]. A limitation of their approach is that by requiring it to be fully automated, it can only certify certain kinds of cost bounds, mostly polynomials, with some exceptions [36].

Work on refinement and intersection types have also been used to statically reason about cost bounds of programs. A notable example is the work of Avanzini et al. [3], where they propose a type system for expected cost analysis of functional programs. However, they did not provide type-checking/inference algorithms to their system, making it unclear how well their approach scales.

Program logics and type theories. The line of work initiated by the cost-aware logical framework [61] provides an elegant approach to reasoning about cost internal to a dependently-typed type theory that uses an open modality for distinguishing between the costful and cost-free aspects of execution. Since then, the authors have proposed [26], an extension of their base type theory. Their main limitation compared to our semantics is that theirs is based on the writer monad transformer. Adapting the semantic ideas of **effcost** to the modal dependently typed setting is an interesting direction for future work.

There has also been work on type theories for relational cost analyses of programs [18, 64]. It is hard to compare their work to **effcost** because they focus on relational reasoning. It is an interesting direction for future work understanding how to extend **effcost** with models for relational reasoning of cost.

Rajani et al. [65, 66] have worked on graded type systems for reasoning both about deterministic cost as well as expected cost. The main conceptual point of departure between their approach and ours is that they define operationally-based denotational semantics. In our understanding, one advantage in favoring purely denotational/categorical semantics is that they are usually closer to mathematics, making it easier to reason about programs using them.

More recently, there have been a few extensions to the Iris program logic that can reason about different notions of cost of effectful programs [1, 29, 55]. Contrary to **effcost**, they base their analysis on an operational semantics and are usually restricted to a fixed cost analysis. Their main advantage about their approach over ours is that, by being implemented in the Iris program logic, their analyses can be integrated into the wider Iris ecosystem. We conjecture that the insights from **effcost** could lead to simpler implementations of program logics for cost analysis.

Denotational approaches. The research programme initiated by Danner et al. [19, 20] has been an inspiration to this paper. In particular, Kavvos et al. [45] used CBPV as a metalanguage for

deterministic recurrence relations. Of course, the main limitation of their approach is that their semantics can only accommodate deterministic notions of cost.

Recently, Azevedo de Amorim [7] has extended the work of Kavvos et al. so that it can also accommodate expected cost analysis. Furthermore, he has proved soundness properties relating his semantics with a monad transformer semantics a CPS semantics and an operational cost semantics. Compared to our own soundness proof, he had to roll-out every logical relation by hand, as opposed to using the fibered approach advocated for by this paper, making the proof significantly longer.

Avanzini et al. [2, 4, 5] has done work on CPS for expected cost and quantum cost. Recently, their approach has been generalized by Kura [48], who proposed a categorical formulation of CPS transformations in functional languages. However, these approaches are limited by characteristics inherent to CPS semantics, as described throughout this paper.

Fibered logical relations. The use of fibrations as a way of organizing proofs by logical relations is a well-established research programme. In the context of monadic effects, there has been much work done in proposing new ways of defining logical relations for them. We highlight the work by Katsumata [42] on $\top\top$ -lifting that was later generalized by the technique of codensity lifting [44]. These techniques have found applications in diverse kinds of logical relations models [39, 43] and have led to interesting mathematical developments to the coalgebraic theory of bisimulation [46, 47]. For future work we want to get a better categorical description of lax fibration

Goubault-Larrecq et al. [24] has also studied semantic accounts of monadic logical relations. The main limitations of their approach is that they do not consider the “fuzzier” notions of monad morphisms such as the ordered monad morphism proposed in this paper. Indeed, none of the existing approaches would have been able to accommodate the soundness proof in Section 5. We imagine that the techniques developed in this paper will have applications to other more quantitative notions of morphisms, metric reasoning being an exciting possibility.

Recent work [8] proposes a fibered characterization of logical relations for CBPV models. While their approach is enough to reconstruct the CPS soundness logical relations, it cannot deal with laxer notions of monad morphisms we have introduced in this paper.

Recurrence relations. There has been much work on using/developing solvers of recurrence relations to aid in the automatic analysis of cost of programs. The main limitation of existing tools/approaches is that they can only handle very limited or non-standard programming languages [17, 51, 72]. We highlight recent work by Rustenholz et al. [68], who have used ideas from domain theory to implement a solver of recurrence relations. It is an interesting direction of future work to study fragments of **effcost** that are well-suited for the use of such automation techniques.

8 CONCLUSION

In this paper we have proposed a semantic analysis of effectful notions of cost. This has been made manifest by using **effcost** as our metalanguage: an effect, cost analysis and evaluation strategy agnostic language. We have justified its soundness by relating our newly proposed categorical semantics to existing semantics from the literature. In order to achieve that, we have advanced the field of categorical logical relations by proposing a new class of fibrations that interact well with order-enriched categories.

REFERENCES

- [1] Alejandro Aguirre, Philipp G Haselwarter, Markus De Medeiros, Kwing Hei Li, Simon Oddershede Gregersen, Joseph Tassarotti, and Lars Birkedal. 2024. Error credits: Resourceful reasoning about error bounds for higher-order probabilistic programs. *Proceedings of the ACM on Programming Languages* ICFP (2024).

- [2] Martin Avanzini, Gilles Barthe, and Ugo Dal Lago. 2021. On continuation-passing transformations and expected cost analysis. In *International Conference on Functional Programming (ICFP)*.
- [3] Martin Avanzini, Ugo Dal Lago, and Alexis Ghyselen. 2019. Type-based complexity analysis of probabilistic functional programs. In *Logic in Computer Science (LICS)*.
- [4] Martin Avanzini, Georg Moser, Romain Péchoux, Simon Perdrix, and Vladimir Zamdzhiev. 2022. Quantum expectation transformers for cost analysis. In *Symposium on Logic in Computer Science (LICS)*.
- [5] Martin Avanzini, Georg Moser, and Michael Schaper. 2020. A modular cost analysis for probabilistic programs. In *Object-oriented Programming, Systems, Languages, and Applications (OOPSLA)*.
- [6] Arthur Azevedo de Amorim, Marco Gaboardi, Justin Hsu, Shin-ya Katsumata, and Ikram Cherigui. 2017. A semantic account of metric preservation. *Principles of Programming Languages (POPL)* (2017).
- [7] Pedro H. Azevedo de Amorim. 2025. Denotational Foundations for Expected Cost Analysis. *Proceedings of the ACM on Programming Languages OOPSLA* (2025).
- [8] Pedro H. Azevedo de Amorim, Satoshi Kura, and Philip Saville. 2025. Logical relations for call-by-push-value models, via internal fibrations in a 2-category. In *Logic in Computer Science (LICS)*.
- [9] Igor Bakovic. 2011. Fibrations of bicategories. *Preprint available at <http://www.irb.hr/korisnici/ibakovic/groth2fib.pdf>* (2011).
- [10] Gilles Barthe, Joost-Pieter Katoen, and Alexandra Silva. 2020. *Foundations of probabilistic programming*. Cambridge University Press.
- [11] Kevin Batz, Adrian Gallus, Benjamin Lucien Kaminski, Joost-Pieter Katoen, and Tobias Winkler. 2022. Weighted programming: a programming paradigm for specifying mathematical models. *Proceedings of the ACM on Programming Languages OOPSLA* (2022). <https://doi.org/10.1145/3527310>
- [12] Kevin Batz, Benjamin Lucien Kaminski, Joost-Pieter Katoen, Christoph Matheja, and Lena Verscht. 2023. A calculus for amortized expected runtimes.
- [13] Jon Beck. 2008. Distributive laws. *Reprints in Theory and Applications of Categories* (2008).
- [14] Francis Borceux. 1994. *Handbook of categorical algebra: volume 2, Categories and Structures*. Vol. 2. Cambridge University Press.
- [15] Mitchell Buckley. 2014. Fibred 2-categories and bicategories. *Journal of Pure and Applied Algebra* (2014).
- [16] Titouan Carrette, Louis Lemonnier, and Vladimir Zamdzhiev. 2023. Central submonads and notions of computation: Soundness, completeness and internal languages. In *Logic in Computer Science (LICS)*. <https://doi.org/10.1109/LICS56636.2023.10175687>
- [17] Krishnendu Chatterjee, Hongfei Fu, Petr Novotný, and Rouzbeh Hasheminezhad. 2016. Algorithmic analysis of qualitative and quantitative termination problems for affine probabilistic programs. In *Principles of Programming Languages (POPL)*.
- [18] Ezgi Çiçek, Gilles Barthe, Marco Gaboardi, Deepak Garg, and Jan Hoffmann. 2017. Relational cost analysis. In *Principles of Programming Languages (POPL)*.
- [19] Norman Danner, Daniel R Licata, and Ramyaa Ramyaa. 2015. Denotational cost semantics for functional languages with inductive types. In *International Conference on Functional Programming (ICFP)*.
- [20] Norman Danner, Jennifer Paykin, and James S Royer. 2013. A static cost analysis for a higher-order language. In *7th workshop on Programming languages meets program verification*.
- [21] Luc Devroye. 1986. A Note on the Height of Binary Search Trees. *Journal of the ACM (JACM)* (1986).
- [22] Andrzej Filinski. 1996. *Controlling Effects*. Ph.D. Dissertation. School of Computer Science, Carnegie Mellon University.
- [23] Gerhard Gierz, Karl Heinrich Hofmann, Klaus Keimel, Jimmie D Lawson, Michael Mislove, and Dana S Scott. 2003. *Continuous Lattices and Domains*. Cambridge University Press.
- [24] Jean Goubault-Larrecq, Sławomir Lasota, and David Nowak. 2008. Logical relations for monadic types. *Mathematical Structures in Computer Science* (2008).
- [25] Alexandre Goy and Daniela Petrişan. 2020. Combining probabilistic and non-deterministic choice via weak distributive laws. In *Logic in Computer Science (LICS)*.
- [26] Harrison Grodin, Yue Niu, Jonathan Sterling, and Robert Harper. 2023. Decalf: A Directed, Effectful Cost-Aware Logical Framework. In *Principles of Programming Languages (POPL)*.
- [27] J. Grosen, D. M. Kahn, and J. Hoffmann. 2023. Automatic Amortized Resource Analysis with Regular Recursive Types. In *Logic in Computer Science (LICS)*.
- [28] Robert Harper. 2016. *Practical Foundations for Programming Languages*. Cambridge University Press.
- [29] Philipp G Haselwarter, Kwing Hei Li, Markus de Medeiros, Simon Oddershede Gregersen, Alejandro Aguirre, Joseph Tassarotti, and Lars Birkedal. 2024. Tachis: Higher-Order Separation Logic with Credits for Expected Costs. *Proceedings of the ACM on Programming Languages OOPSLA2* (2024).
- [30] Claudio Alberto Hermida. 1993. *Fibrations, Logical Predicates and Indeterminates*. Ph.D. Dissertation. The University of Edinburgh.

- [31] Tony Hoare, Bernhard Möller, Georg Struth, and Ian Wehrman. 2011. Concurrent Kleene algebra and its foundations. *The Journal of Logic and Algebraic Programming* (2011).
- [32] Jan Hoffmann, Ankush Das, and Shu-Chun Weng. 2017. Towards automatic resource bound analysis for OCaml. In *Symposium on Principles of Programming Languages (POPL)*.
- [33] Jan Hoffmann and Zhong Shao. 2015. Automatic static cost analysis for parallel programs. In *European Symposium on Programming (ESOP)*.
- [34] Mathieu Huot, Alexander K Lew, Vikash K Mansinghka, and Sam Staton. 2023. ω pap spaces: Reasoning denotationally about higher-order, recursive probabilistic and differentiable programs. In *2023 38th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. IEEE, 1–14.
- [35] Peter T. Johnstone. 2002. *Sketches of an Elephant: A Topos Theory Compendium, Volume 1*. Oxford University Press.
- [36] David M Kahn and Jan Hoffmann. 2020. Exponential automatic amortized resource analysis. In *Foundations of Software Science and Computation Structures (FoSSaCS)*.
- [37] Benjamin Lucien Kaminski and Joost-Pieter Katoen. 2017. A weakest pre-expectation semantics for mixed-sign expectations. In *Logic in Computer Science (LICS)*.
- [38] Benjamin Lucien Kaminski, Joost-Pieter Katoen, Christoph Matheja, and Federico Olmedo. 2016. Weakest precondition reasoning for expected run-times of probabilistic programs. In *European Symposium on Programming (ESOP)*.
- [39] Ohad Kammar, Shin-ya Katsumata, and Philip Saville. 2022. Fully abstract models for effectful λ -calculi via category-theoretic logical relations. *Proceedings of the ACM on Programming Languages* POPL (2022).
- [40] Ohad Kammar and Dylan McDermott. 2018. Factorisation systems for logical relations and monadic lifting in type-and-effect system semantics. *Electronic notes in theoretical computer science* (2018).
- [41] Tobias Kappé, Paul Brunet, Alexandra Silva, and Fabio Zanasi. 2018. Concurrent Kleene algebra: Free model and completeness. In *European Symposium on Programming (ESOP)*.
- [42] Shin-ya Katsumata. 2005. A semantic formulation of $\top\top$ -lifting and logical predicates for computational metalanguage. In *Computer Science Logic (CSL)*.
- [43] Shin-ya Katsumata. 2013. Relating computational effects by $\top\top$ -lifting. *Information and Computation* (2013).
- [44] Shin-ya Katsumata, Tetsuya Sato, and Tarmo Uustalu. 2018. Codensity lifting of monads and its dual. *Logical Methods in Computer Science* (2018).
- [45] GA Kavvos, Edward Morehouse, Daniel R Licata, and Norman Danner. 2019. Recurrence extraction for functional programs through call-by-push-value. In *Principles of Programming Languages (POPL)*.
- [46] Yuichi Komorida, Shin-ya Katsumata, Nick Hu, Bartek Klin, and Ichiro Hasuo. 2019. Codensity Games for Bisimilarity. In *Symposium on Logic in Computer Science (LICS)*.
- [47] Mayuko Kori, Kazuki Watanabe, Jurriaan Rot, and Shin-ya Katsumata. 2024. Composing Codensity Bisimulations. In *Symposium on Logic in Computer Science (LICS)*.
- [48] Satoshi Kura. 2023. Higher-Order Weakest Precondition Transformers via a CPS Transformation. *arXiv preprint arXiv:2301.09997* (2023).
- [49] Satoshi Kura, Natsuki Urabe, and Ichiro Hasuo. 2019. Tail probabilities for randomized program runtimes via martingales for higher moments. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*.
- [50] Jim Laird, Giulio Manzonetto, Guy McCusker, and Michele Pagani. 2013. Weighted Relational Models of Typed Lambda-Calculi. In *Logic in Computer Science (LICS)*. <https://doi.org/10.1109/LICS.2013.36>
- [51] Lorenz Leutgeb, Georg Moser, and Florian Zuleger. 2022. Automated expected amortised cost analysis of probabilistic data structures. In *Computer Aided Verification (CAV)*.
- [52] Paul Blain Levy. 2001. *Call-by-push-value*. Ph.D. Dissertation. Queen Mary University of London.
- [53] Alexander K Lew, Mathieu Huot, Sam Staton, and Vikash K Mansinghka. 2023. ADEV: Sound automatic differentiation of expected values of probabilistic programs. *Proceedings of the ACM on Programming Languages* POPL (2023).
- [54] Benjamin Lichtman and Jan Hoffmann. 2017. Arrays and references in resource aware ML. In *Formal Structures for Computation and Deduction (FSCD 2017)*.
- [55] Glen Mével, Jacques-Henri Jourdan, and François Pottier. 2019. Time credits and time receipts in Iris. In *European Symposium on Programming (ESOP)*.
- [56] Matteo Mio. 2014. Upper-expectation bisimilarity and Łukasiewicz μ -calculus. In *Foundations of Software Science and Computation Structures (FoSSaCS)*.
- [57] John C Mitchell and Andre Scedrov. 1992. Notes on scoping and relators. In *International Workshop on Computer Science Logic*.
- [58] E Moggi. 1989. Computational lambda-calculus and monads. In *Logic in Computer Science (LICS)*.
- [59] Van Chan Ngo, Quentin Carbonneaux, and Jan Hoffmann. 2018. Bounded expectations: resource analysis for probabilistic programs. In *Programming Language Design and Implementation (PLDI)*.
- [60] Hanne Riis Nielson. 1987. A Hoare-like proof system for analysing the computation time of programs. *Science of Computer Programming* (1987).

- [61] Yue Niu, Jonathan Sterling, Harrison Grodin, and Robert Harper. 2022. A cost-aware logical framework. In *Principles of Programming Languages (POPL)*.
- [62] Hugo Paquet and Philip Saville. 2024. Effectful semantics in bicategories: strong, commutative, and concurrent pseudomonads. In *Symposium on Logic in Computer Science (LICS)*. <https://doi.org/10.1145/3661814.3662130>
- [63] Gordon Plotkin, John Power, Donald Sannella, and Robert Tennent. 2000. Lax logical relations. In *International Colloquium on Automata, Languages, and Programming (ICALP)*.
- [64] Weihao Qu, Marco Gaboardi, and Deepak Garg. 2019. Relational cost analysis for functional-imperative programs. *Proceedings of the ACM on Programming Languages* ICFP (2019).
- [65] Vineet Rajani, Gilles Barthe, and Deepak Garg. 2024. A Modal Type Theory of Expected Cost in Higher-Order Probabilistic Programs. *Proceedings of the ACM on Programming Languages* OOPSLA (2024).
- [66] Vineet Rajani, Marco Gaboardi, Deepak Garg, and Jan Hoffmann. 2021. A unifying type-theory for higher-order (amortized) cost analysis. In *Symposium on Principles of Programming Languages (POPL)*.
- [67] Exequiel Rivas and Tarmo Uustalu. 2024. Concurrent monads for shared state. In *Principles and Practice of Declarative Programming (PPDP)*.
- [68] Louis Rustenholz, Pedro López-García, José F Morales, and Manuel V Hermenegildo. 2024. An Order Theory Framework of Recurrence Equations for Static Cost Analysis—Dynamic Inference of Non-Linear Inequality Invariants. In *Static Analysis Symposium (SAS)*.
- [69] Gavin J. Seal. 2013. Tensors, Monads and Actions. *Theory and Applications of Categories* (2013).
- [70] Jesse Sigal. 2024. *Automatic Differentiation via Effects and Handlers*. Ph. D. Dissertation. University of Edinburgh.
- [71] Jonathan Sterling and Robert Harper. 2021. Logical relations as types: Proof-relevant parametricity for program modules. *Journal of the ACM (JACM)* (2021).
- [72] Yican Sun, Hongfei Fu, Krishnendu Chatterjee, and Amir Kafshdar Goharshady. 2023. Automated Tail Bound Analysis for Probabilistic Recurrence Relations. In *Computer Aided Verification (CAV)*.
- [73] Matthijs Vákár, Ohad Kammar, and Sam Staton. 2019. A Domain Theory for Statistical Probabilistic Programming. *Proceedings of the ACM on Programming Languages* POPL (2019).
- [74] Daniele Varacca and Glynn Winskel. 2006. Distributing probability over non-determinism. *Mathematical Structures in Computer Science* (2006).
- [75] Di Wang, David M Kahn, and Jan Hoffmann. 2020. Raising expectations: automating expected cost analysis with types. *International Conference on Functional Programming (ICFP)*.

$$\begin{array}{c}
\frac{(x, \tau) \in \Gamma}{\Gamma \vdash_v x : \tau} \quad \frac{\Gamma \vdash_v V_1 : \tau_1 \quad \Gamma \vdash_v V_2 : \tau_2}{\Gamma \vdash_v (V_1, V_2) : \tau_1 \times \tau_2} \quad \frac{\Gamma \vdash_v V_i : \tau_i \quad i \in \{1, 2\}}{\Gamma \vdash_v \text{in}_i V_i : \tau_1 + \tau_2} \quad \frac{\Gamma \vdash_c t : \bar{\tau}}{\Gamma \vdash_v \text{thunk } t : U\bar{\tau}} \\
\\
\frac{}{\Gamma \vdash_v b : B} \quad \frac{}{\Gamma \vdash_v a : \mathbb{A}} \quad \frac{}{\Gamma \vdash_v () : 1} \\
\\
\frac{\Gamma \vdash_v V : \tau\{\mu\alpha. \tau/\alpha\}}{\Gamma \vdash_c \tau.\text{roll } V : \mu\alpha. \tau}
\end{array}$$

Fig. 6. Value typing judgments

$$\begin{array}{c}
\frac{\Gamma \vdash_v V : U\bar{\tau}}{\Gamma \vdash_c \text{force } V : \bar{\tau}} \quad \frac{\Gamma, x : \tau \vdash_c t : \bar{\tau}}{\Gamma \vdash_c \lambda x. t : \tau \rightarrow \bar{\tau}} \quad \frac{\Gamma \vdash_v V : \tau \quad \Gamma \vdash_c t : \tau \rightarrow \bar{\tau}}{\Gamma \vdash_c t V : \bar{\tau}} \\
\\
\frac{\Gamma \vdash_c t_1 : \bar{\tau}_1 \quad \Gamma \vdash_c t_2 : \bar{\tau}_2}{\Gamma \vdash_c t_1 \& t_2 : \bar{\tau}_1 \times \bar{\tau}_2} \quad \frac{\Gamma \vdash_c t : \bar{\tau}_i \times \bar{\tau}_2}{\Gamma \vdash_c \pi_i t : \bar{\tau}_i} \quad \frac{\Gamma \vdash_v V : \tau_1 \times \tau_2 \quad \Gamma, x_1 : \tau_1, x_2 : \tau_2 \vdash_v t : \bar{\tau}}{\text{let } (x_1, x_2) = V \text{ in } t : \bar{\tau}} \\
\\
\frac{\Gamma \vdash_v V : \tau_1 + \tau_2 \quad \Gamma, x : \tau_1 \vdash_c t : \bar{\tau} \quad \Gamma, x : \tau_2 \vdash_c u : \bar{\tau}}{\Gamma \vdash_c \text{case } V \text{ of } (| \text{in}_1 x \Rightarrow t \mid \text{in}_2 x \Rightarrow u) : \bar{\tau}} \\
\\
\frac{\Gamma \vdash_c t : \tau\{\mu\alpha. \tau/U\bar{\tau}\} \rightarrow \bar{\tau}}{\Gamma \vdash_c \tau.\text{fold } t : \mu\alpha. \tau \rightarrow \bar{\tau}} \quad \frac{\Gamma \vdash_v V : \mu\alpha. \tau \quad \Gamma, x : \tau\{\mu\alpha. \tau/\alpha\} \vdash_c t : \bar{\tau}}{\Gamma \vdash_c \text{match } V \text{ with roll } x \rightarrow t : \bar{\tau}} \quad \frac{\text{op} \in O(\tau_1, \tau_2)}{\Gamma \vdash_c \text{op} : \tau_1 \rightarrow F\tau_2} \\
\\
\frac{\Gamma \vdash_v V : \mathbb{A}}{\Gamma \vdash_v \text{charge } V : F1} \quad \frac{\Gamma \vdash_v V : \tau}{\Gamma \vdash_v \text{produce } V : F\tau} \quad \frac{\Gamma \vdash_c t : F\tau \quad \Gamma, x : \tau \vdash_c u : \bar{\tau}}{\Gamma \vdash_c x \leftarrow t; u : \bar{\tau}}
\end{array}$$

Fig. 7. Computation typing judgement

A effcost: TYPING RULES AND CATEGORICAL SEMANTICS

A.1 Typing Rules

The typing rules are standard for presentations of CBPV extended with positive inductive types. We omit the usual kinding judgements, as they are standard.

A.2 Categorical Semantics

The categorical semantics is also standard and follows directly by the universal property of the connectives and is defined by induction of the typing derivations.

Definition A.1. Given two T -algebras (A, α) and (B, β) , their product is given by $(A \times B, (\alpha \times \beta) \circ \langle T\pi_1, T\pi_2 \rangle)$

Definition A.2. If $A : C$ is an object of the base category then $(TA, (id_A)^\#)$ is the *free* T -algebra.

The thunk and force operations are invisible in this model. For λ -abstraction we use the Cartesian closed isomorphism $A \times B \rightarrow C \cong A \rightarrow B \rightarrow C$. For application, we use the counit of the Cartesian closed adjunction $A \times (A \rightarrow B) \rightarrow B$. For the lazy product types, we use the Cartesian structure of

the category: the introduction rule is defined by the universal property of Cartesian product, the elimination rule are the projection morphisms that exist in Cartesian categories.

The same is true about the product and sum value types. Note that for the interpretation of the sum type we need to use the isomorphism $\Gamma \times (A + B) \cong \Gamma \times A + \Gamma \times B$, which exists because $A \times -$ is a left adjoint and therefore preserves colimits. The produce operator is interpreted as the unit of the monad while the sequencing $x \leftarrow t; u$, assuming that the output type has algebra structure α , is interpreted as

$$\Gamma \xrightarrow{\langle \Gamma, t \rangle} \Gamma \times UF\tau \xrightarrow{\sigma} UF(\Gamma \times \tau) \xrightarrow{UFu} UFU\bar{\tau} \xrightarrow{\alpha} U\bar{\tau}$$

Finally, the inductive types are interpreted using the fact that their semantics is given by initial algebras. In particular, roll is the initial algebra isomorphism. fold is the morphism given by initiality and match is just sequential composition.

B PROOF OF THEOREM 4.14

Before we can prove this theorem we have to establish some of the basic theory of lax fibrations. We begin by generalizing an important theorem from the theory of fibrations which shows that they are closed under exponentiation.

Theorem B.1. *If $p : \mathcal{E} \rightarrow \mathcal{C}$ is a lax fibration then for every Poset-category X , $p \circ - : [X, \mathcal{E}] \rightarrow [X, \mathcal{C}]$ is a lax fibration.*

PROOF. We can equip the Poset-functor categories $[X, \mathcal{E}]$ and $[X, \mathcal{C}]$ with canonical pointwise Poset-enriched structures: given two natural transformations $\alpha, \beta : F \rightarrow G$, we say that $\alpha \leq \beta$ if for every $A : X$, $\alpha_A \leq \beta_A$.

In order to show that $p \circ -$ is indeed a lax fibration, we first prove that it is locally order reflective. This follows from the order of natural transformations being given pointwise and by p being locally order reflective. Next, we want to show that it satisfies the morphism lifting situation:

$$\begin{array}{ccc}
 \hat{H} & \xrightarrow{\quad \leq \quad} & \hat{G} \\
 \downarrow & \searrow & \downarrow \\
 \hat{F} & \xrightarrow{\quad} & \hat{G}
 \end{array}$$

$$\begin{array}{ccc}
 H \circ p & \xrightarrow{\quad \gamma \quad} & G \circ p \\
 \downarrow \beta & \searrow \delta & \downarrow \alpha \\
 F \circ p & \xrightarrow{\quad} & G \circ p
 \end{array}$$

First, we have to define the functor $\hat{F}$. Its action on objects is given by using the componentwise lax fibration structure of p . This will give us objects $\hat{F}(A)$ and morphisms $\hat{\alpha}_A : F(A) \rightarrow G(A)$. In order to prove that $\hat{\alpha}$ is indeed a natural transformation, we use the fact that α_p is a natural transformation and p is locally order reflective.

We conclude the proof by showing that $\hat{\alpha}$ does indeed satisfy the appropriate universal property. Assuming the existence of natural transformations β and γ , the lift $\hat{\beta}$ is once again defined componentwise and it is indeed a natural transformation by the 2-cell part of the universal property of p . We show that $\hat{\beta}$ satisfies the 2-cell part of the universal property by using the fact that p is a lax fibration and the order structure of natural transformations being given componentwise. $\square$

In the context of fibrations for logical relations, it is possible to construct new fibrations for logical relations from old ones by using the theorem that fibrations are stable under pullback, as shown in Example 4.4. We now show that this property also generalizes to the lax case as well.

Theorem B.2. *Lax fibrations are stable under pullback in PosCat.*

PROOF. Pullbacks in **PosCat** are calculated just like in **Cat**. Therefore, let $p : \mathcal{E} \rightarrow \mathcal{B}$ be a lax fibration and $F : \mathcal{C} \rightarrow \mathcal{B}$ a **Poset**-functor. The pullback p^*F is the following **Poset**-category:

- Objects are pairs $(X : \mathcal{E}, A : \mathcal{C})$ such that $p(X) = F(A)$.
- Morphisms $(X, A) \rightarrow (Y, B)$ are pairs of morphisms $f : X \rightarrow Y$ and $g : A \rightarrow B$ such that $p(f) = F(g)$.
- The **Poset**-enrichment structure is given by $(f_1, g_1) \leq (f_2, g_2)$ if, and only if, $p(f_1) \leq p(f_2)$.

We now prove that $p^*F \rightarrow \mathcal{C}$ is indeed a lax fibration.

Locally order reflective Let (f_1, g_1) and (f_2, g_2) be a pair of morphisms such that $g_1 \leq g_2$. Since the functor F is monotonic, $F(g_1) \leq F(g_2)$. We conclude by noting that, by definition of morphisms in p^*F , $F(g_i) = p(f_i)$ and, therefore, $(f_1, g_1) \leq (f_2, g_2)$.

Existence Let $f : A \rightarrow B$ be a $\mathcal{C}$ morphism and (Y, B) a p^*F object. Its Cartesian lifting is the morphism $(g, f) : (X, A) \rightarrow (Y, B)$, where X and g are defined using the Cartesian lifting of $F(f)$ with respect to Y , giving us an $\mathcal{E}$ morphism $g : X \rightarrow Y$ above $F(f)$.

Universal property Next, we prove that (g, f) does satisfy the appropriate universal property. The proof follows analogously to the non-lax case, where we use the functor F to define the $\mathcal{E}$ component of p^*F . Since F is monotonic and p is assumed to be a lax fibration we can define the f_1 below.

$$\begin{array}{ccc}
 Z & \xrightarrow{f_2} & Y \\
 \searrow f_1 & \leq & \nearrow \\
 & X \xrightarrow{f} & Y
 \end{array}$$

$$\begin{array}{ccc}
 F(C) & \xrightarrow{Fg_2} & F(B) \\
 \searrow Fg_1 & \leq & \nearrow \\
 & F(A) \xrightarrow{Fg} & F(B)
 \end{array}$$

Now, assume that $(f'_2, g'_2) \leq (f_2, g_2)$, $g \circ g'_1 \leq f'_1$ and $g'_1 \leq g_1$. We conclude the proof by showing that $(f'_1, g'_1) \leq (f_1, g_1)$. By definition, it suffices to prove that $F(g'_1) \leq F(g_1)$, which follows by monotonicity of F .

□

PROOF. Let $S : \mathcal{C} \rightarrow \mathcal{C}$ and $T : \mathcal{D} \rightarrow \mathcal{D}$ be two lax ordered monads; $q : \mathcal{F} \rightarrow \mathcal{D}$ a lax fibration; and $\hat{T} : \mathcal{F} \rightarrow \mathcal{F}$ a lifting of T . Assuming that $(F, \lambda) : \mathcal{C} \rightarrow \mathcal{D}$ is a lax monad morphism, we want to find its Cartesian lifting. We begin by taking the following pullback.

$$\begin{array}{ccc}
 \mathcal{E} & \xrightarrow{\hat{F}} & \mathcal{F} \\
 p \downarrow & \lrcorner & \downarrow q \\
 \mathcal{C} & \xrightarrow{F} & \mathcal{D}
 \end{array}$$

The lifted monad $\hat{S}$ will be a monad over $\mathcal{E}$.

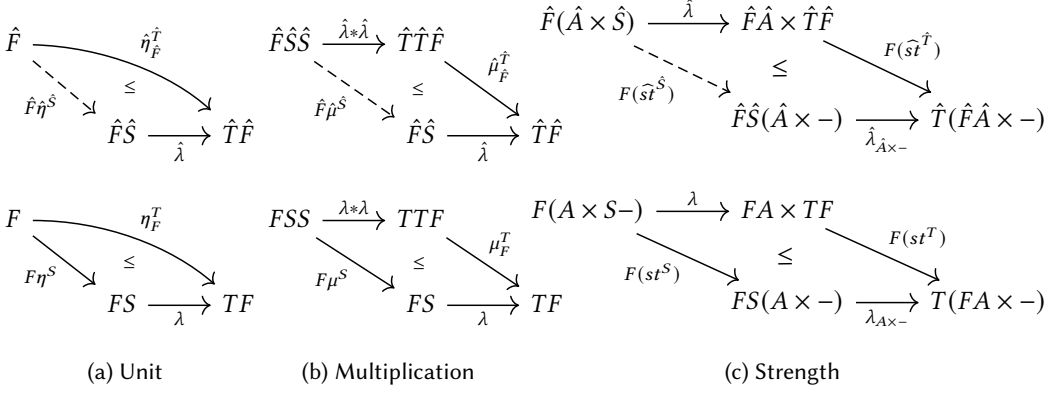


Fig. 8. Lifted monad structure

Since for every **Poset**-category $\mathcal{X}$, $[\mathcal{X}, -]$ is a right adjoint, it preserves limits. Therefore we have the following pullback square where each vertical leg is a lax fibration by Theorem B.2.

$$\begin{array}{ccc} [\mathcal{X}, \mathcal{E}] & \longrightarrow & [\mathcal{X}, \mathcal{F}] \\ \downarrow & \lrcorner & \downarrow \\ [\mathcal{X}, \mathcal{C}] & \longrightarrow & [\mathcal{X}, \mathcal{D}] \end{array}$$

By using the lax fibration structure of $[\mathcal{E}, p]$, we can define the functor $\hat{G}$ and natural transformation $\hat{\lambda}$.

$$\hat{G} \xrightarrow{\hat{\lambda}} \hat{F}\hat{T}$$

$$SF \xrightarrow{\lambda} FT$$

Next, we want to factor $\hat{G}$ through $\hat{F}$, so that we obtain an endofunctor $\hat{S} : \hat{E} \rightarrow \hat{E}$ above S . This is achieved by using the universal property of pullbacks.

$$\begin{array}{ccccc} \mathcal{E} & & \hat{G} & & \\ \downarrow \hat{S} & \searrow & & \searrow & \\ \mathcal{E} & \xrightarrow{\hat{F}} & \mathcal{F} & & \\ \downarrow p & \lrcorner & \downarrow q & & \\ \mathcal{C} & \xrightarrow{F} & \mathcal{D} & & \end{array}$$

Sp is the arrow from $\mathcal{E}$ to $\mathcal{C}$.

Now we have to equip $\hat{S}$ with its strong monad structure, which is defined using the universal property of Cartesian liftings, as depicted in Figure 8. Note that, for example, in the unit example, the universal property will only give us a natural transformation $\alpha : \hat{F} \rightarrow \hat{F}\hat{S}$ and not necessarily one of the shape $\hat{F}\hat{\eta}^{\hat{S}}$. In order to factor α through $\hat{F}$ we have to once again use the universal property of pullbacks.

Universal Property. Now, consider a third monad lifting $(M, \hat{M})$ with respect to a fibration $q' : \mathcal{E}' \rightarrow \mathcal{C}'$ as follows

$$\begin{array}{ccc}
 \hat{M} & \xrightarrow{(\hat{H}, \hat{\tau})} & \hat{T} \\
 & \searrow & \downarrow (\hat{F}, \hat{\lambda}) \\
 & \hat{S} & \\
 M & \xrightarrow{(H, \tau)} & T \\
 & \searrow (G, \beta) & \downarrow (F, \lambda) \\
 & S & \\
 & & T
 \end{array}$$

Note that, since we are trying to prove that the functor is a fibration, not a lax fibration, we have to assume that the downstairs triangle commutes. We have to show that there is a unique $(\hat{G}, \hat{\beta}) : \hat{M} \rightarrow \hat{S}$ satisfying the appropriate conditions. The functor $\hat{G}$ is defined by the universal property of pullbacks.

$$\begin{array}{ccccc}
 \mathcal{E}' & & \xrightarrow{\hat{H}} & & \mathcal{F} \\
 q' \downarrow & \dashrightarrow \hat{G} & & \downarrow \hat{F} & \\
 \mathcal{C}' & & \xrightarrow{G} & & \mathcal{C} \\
 & & & & \downarrow q \\
 & & & & \mathcal{D} \\
 & & & & \downarrow p \\
 & & & & \mathcal{D}
 \end{array}$$

The lifted natural transformation $\hat{\beta}$ is defined by the universal property of Cartesian liftings. The lifted triangle commutes since lax fibrations are assumed to locally reflect the order.

$$\begin{array}{ccc}
 \hat{F}\hat{G}\hat{M} & \xrightarrow{\hat{\tau}} & \hat{T}\hat{F}\hat{G} \\
 \hat{F}\hat{\beta} \dashrightarrow & & \downarrow \hat{\lambda}_G \\
 \hat{F}\hat{S}\hat{G} & \xrightarrow{\hat{\lambda}_G} & \hat{T}\hat{F}\hat{G} \\
 \\
 FGM & \xrightarrow{\tau} & TFG \\
 F\beta \searrow & & \downarrow \lambda_G \\
 FSG & \xrightarrow{\lambda_G} & TFG
 \end{array}$$

In order to show that β is indeed an ordered monad morphism, we use the fact that lax fibrations are locally order reflective. Uniqueness of $(\hat{G}, \hat{\beta})$ follows from the universal property of pullbacks. This concludes the proof. $\square$

C PROOF OF LEMMA 5.4

PROOF. We equip $\mathbb{A}$ with the following algebra structure $\mathbb{A} \times T\mathbb{A} \xrightarrow{\mathbb{A} \times \alpha} \mathbb{A} \times \mathbb{A} \xrightarrow{+} \mathbb{A}$. We now prove that it satisfies the algebra axioms, starting from the unit law and then the multiplication

law.

$$\begin{array}{c}
 \begin{array}{ccc}
 \mathbb{A} & \xrightarrow{0 \times \eta_{\mathbb{A}}} & \mathbb{A} \times T\mathbb{A} \\
 \searrow 0 \times id_{\mathbb{A}} & & \downarrow \mathbb{A} \times \alpha \\
 & & \mathbb{A} \times \mathbb{A} \\
 \searrow id_{\mathbb{A}} & & \downarrow + \\
 & & \mathbb{A}
 \end{array} \\
 \\
 \begin{array}{ccccccc}
 \mathbb{A} \times T(\mathbb{A} \times T(\mathbb{A})) & \xrightarrow{\mathbb{A} \times \langle T\pi_1, T\pi_2 \rangle} & \mathbb{A} \times T(\mathbb{A}) \times T^2(\mathbb{A}) & \xrightarrow{\mathbb{A} \times \alpha \times \mu} & \mathbb{A} \times \mathbb{A} \times T(\mathbb{A}) & \xrightarrow{add \times T(\mathbb{A})} & \mathbb{A} \times T(\mathbb{A}) \\
 \downarrow \mathbb{A} \times T(\mathbb{A} \times \alpha) & & & & & & \downarrow \mathbb{A} \times \alpha \\
 \mathbb{A} \times T(\mathbb{A} \times \mathbb{A}) & \xrightarrow{\mathbb{A} \times \langle T\pi_1, T\pi_2 \rangle} & \mathbb{A} \times T(\mathbb{A}) \times T(\mathbb{A}) & \xrightarrow{\mathbb{A} \times \alpha \times T(\mathbb{A})} & (\mathbb{A} \times \mathbb{A}) \times T(\mathbb{A}) & \xrightarrow{add \times \alpha} & \mathbb{A} \times \mathbb{A} \\
 \downarrow \mathbb{A} \times Tadd & & \downarrow \mathbb{A} \times \alpha \times \alpha & & & & \downarrow add \\
 & & \mathbb{A} \times \mathbb{A} \times \mathbb{A} & & & & \\
 & & \downarrow \mathbb{A} \times add & & & & \\
 \mathbb{A} \times T(\mathbb{A}) & \xrightarrow{\mathbb{A} \times \alpha} & \mathbb{A} \times \mathbb{A} & \xrightarrow{add} & \mathbb{A} & &
 \end{array}
 \end{array}$$

□

D PROOF OF LEMMA 5.10

PROOF. There are two properties it must satisfy. The first one is the unit coherence condition:

$$\begin{array}{ccc}
 1 & & \\
 \langle 0, id \rangle \downarrow & \searrow \langle 0, \eta \rangle & \\
 \mathbb{A} \times - & & \mathbb{A} \times T- \\
 \eta \downarrow & \nearrow \langle \alpha \circ T\pi_1, T\pi_2 \rangle & \\
 T(\mathbb{A} \times -) & &
 \end{array}$$

It commutes by assumption that α is an algebra structure, so it cancels the unit η and by naturality of η . The multiplication coherence condition is a bit harder to prove:

$$\begin{array}{ccccccc}
 T(\mathbb{A} \times T\mathbb{A}) & \xrightarrow{\langle T\pi_1, T\pi_2 \rangle} & T\mathbb{A} \times T^2\mathbb{A} & \xrightarrow{T\mathbb{A} \times \mu} & T\mathbb{A} \times T\mathbb{A} & \xrightarrow{\alpha \times \alpha} & \mathbb{A} \times \mathbb{A} \\
 \downarrow T(\mathbb{A} \times \alpha) & & & & \downarrow \alpha \times \alpha & & \downarrow + \\
 T(\mathbb{A} \times \mathbb{A}) & & & & T\mathbb{A} \times \mathbb{A} & \xrightarrow{\alpha} & \mathbb{A} \\
 \downarrow \langle T\pi_1, T\pi_2 \rangle & & & & \downarrow \alpha \times \alpha & & \\
 T\mathbb{A} \times T\mathbb{A} & \xrightarrow{\alpha \times \alpha} & \mathbb{A} \times \mathbb{A} & \xrightarrow{+} & \mathbb{A} & &
 \end{array}$$

(strongly effectful) μ (naturality) μ

(Alg. axiom) $\alpha \times \alpha$ (Alg. map)

The proofs that the subdiagrams (Alg. axiom), (Alg. map) and (naturality) commute is straightforward. The proof that the subdiagram (strongly effectful) commutes should be done component wise. The first component is easy while the second one requires the strongly effectful assumption. $\square$

E PROOF OF LEMMA 5.19

We will need the following useful lemma.

Lemma E.1 (cf. Johnstone [35, Proposition 4.3.1]). *Let $\mathcal{E}$ be a Cartesian closed category, and $\mathcal{L}$ a reflective subcategory of $\mathcal{E}$. If the reflector $\mathcal{E} \rightarrow \mathcal{L}$ preserves products then $\mathcal{L}$ is Cartesian closed as well.*

We now prove Lemma 5.19.

PROOF. First we have to prove that $\mathbf{Pred}_{dc}$ is bicartesian closed. We want to prove that by using the proposition above.

- The category $\mathbf{Pred}$ is Cartesian closed [57, Proposition 1]
- The category $\mathbf{Pred}_{dc} \hookrightarrow \mathbf{Pred}$ is a reflective subcategory by observing that the this inclusion is right adjoint to the functor that maps $(A, \mathcal{X})$ to $(A, \downarrow \mathcal{X})$, where $\downarrow \mathcal{X}$ is the downward closure of $\mathcal{X}$. Furthermore, a direct calculation shows that this functor preserves Cartesian products.

Now, it suffices to show that U strictly preserves the biCCC structure. This is a direct consequence of $\mathbf{Pred} \rightarrow \mathcal{C}$ being a logical relations fibration and the adjunction between $\mathbf{Pred}_{dc}$ and $\mathbf{Pred}$ preserving the carrier set.

We conclude the proof by referring to Example 4.10 that shows that U is indeed a lax fibration. $\square$